MHEDAS
MASTER HEALTH DATA SCIENCE

Master's Degree in Health Data Science

MHEDAS : *Biomedical sensors and signal processing - 2025/2026*

MHEDAS
Master Health Data Science

eit Health

Co-funded by the European Union

Universitat Rovira i Virgili

UGA
Université Grenoble Alpes

BIOINFORMATICS BARCELONA

AMGEN

UPC
Universitat Politècnica de Catalunya
BarcelonaTech

Universitat de Barcelona

UAB
Universitat Autònoma de Barcelona

Universitat de Girona

Universitat de Lleida

UVIC
Universitat de Vic
Universitat Central de Catalunya

Laboratory 1 Report
Group E

Authors:
Carmen Aznar Mathonneau; Kamela Xhengo; Marco Russo; Yuxi Qiao.

Laboratory 1 Report

- Task 1 – Schematics of a mixed signal circuit.
- Task 2 – Programming the Arduino board.
- Task 3 – Introducing the LCD display.
- Task 4 – Conditioning circuit.
- Task 5 – Results.

Laboratory 1 Report

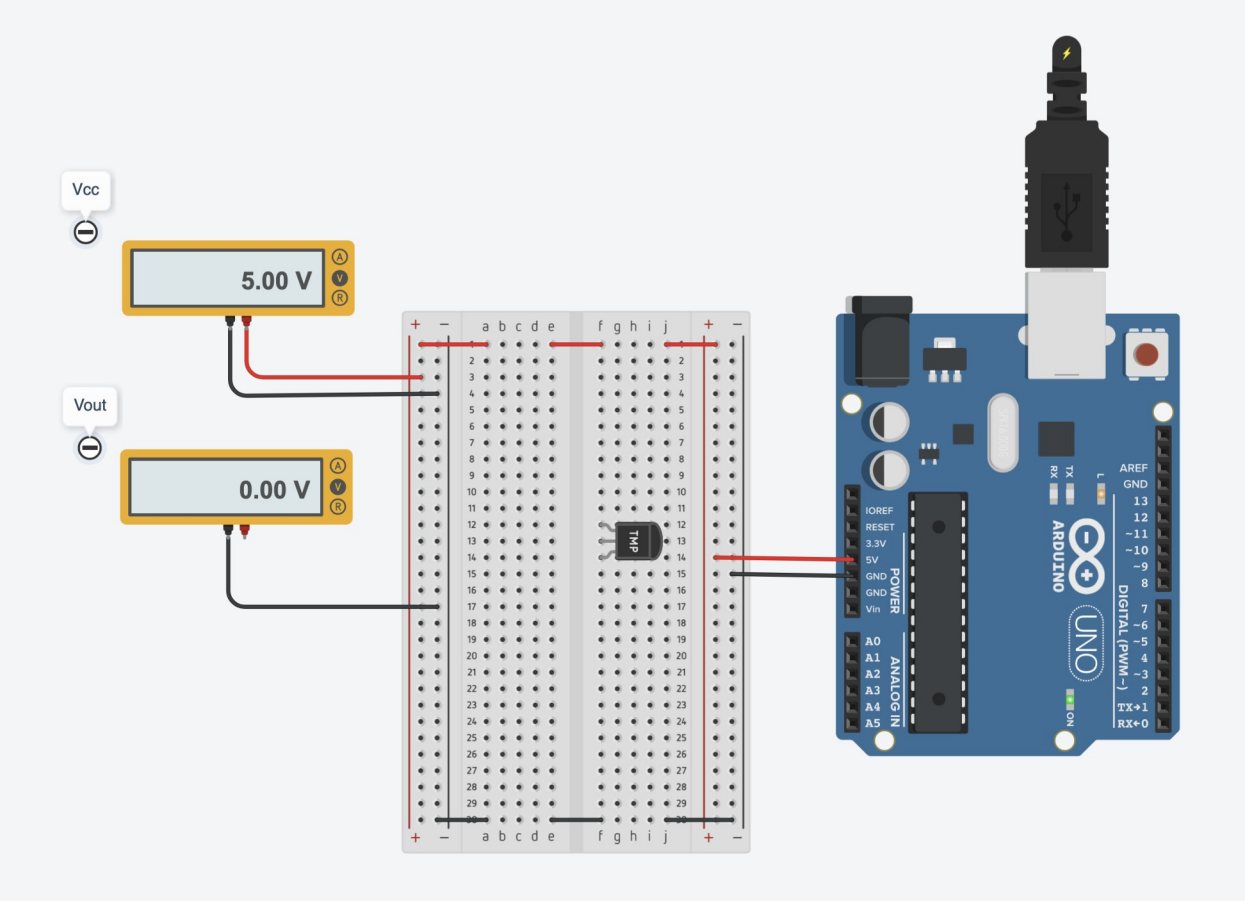
- **Task 1 – Schematics of a mixed signal circuit.**
- Task 2 – Programming the Arduino board.
- Task 3 – Introducing the LCD display.
- Task 4 – Conditioning circuit.
- Task 5 – Results.

Lab1: Task 1

Table II

Group	Sensor	Resistive load	Sensor input range
Group A	Force	10 kΩ	From 1.0 N to 5.0 N
Group B	Temperature	No required	From 10 ºC to 50 ºC
Group C	Flex	100 kΩ	From 0 º to 160 º
Group D	Force	10 kΩ	From 2 N to 9 N
Group E	Temperature	No required	From 0 ºC to 80 ºC
Group F	Flex	100 kΩ	From 20 º to 120 º
Group G	Temperature	No required	From 5 ºC to 55 ºC

Lab1: Task 1. Introducing a mixed signal circuit design.

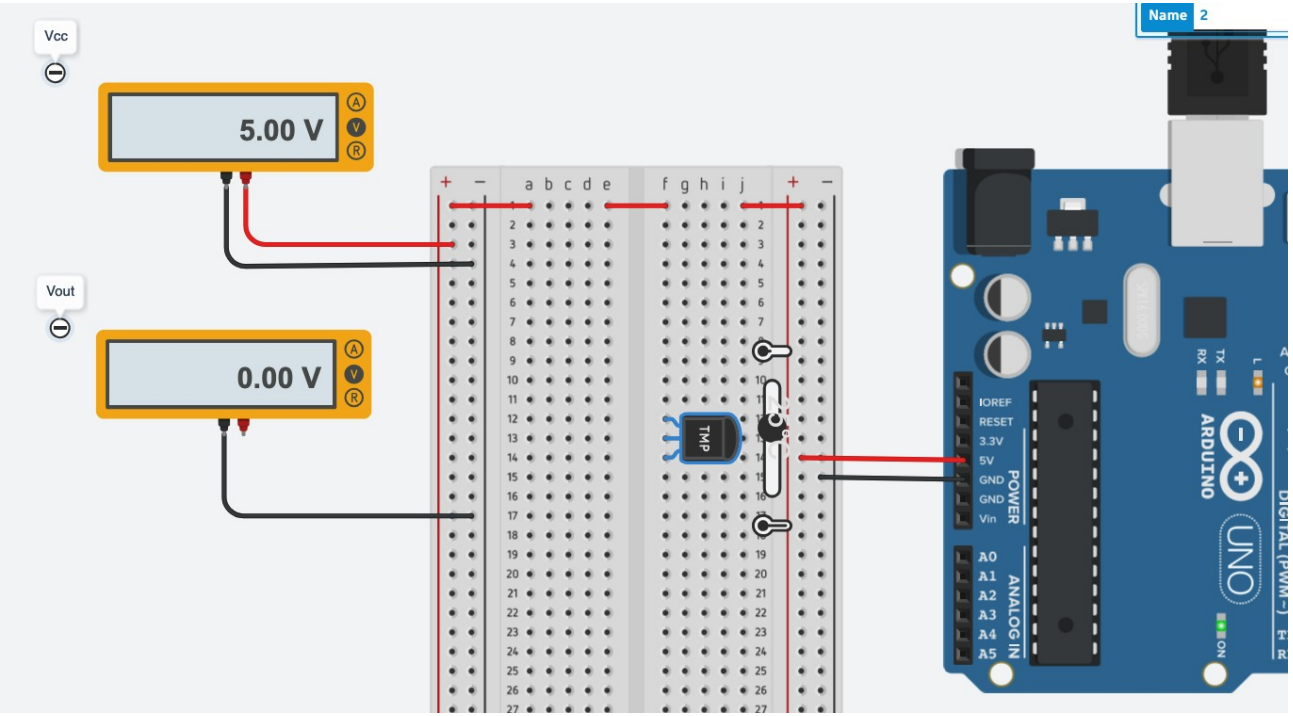


In the first task we introduce a simple circuit design with following components:

- 1x Arduino Uno board
- 1x Breadboard small
- 1x Temperature sensors TMP36
- 2x Multimeters

The temperature sensor using breadboard. The first voltmeter show the input voltage, 5V. The second one, is off, because the analog signal is simulated and not wired as well.

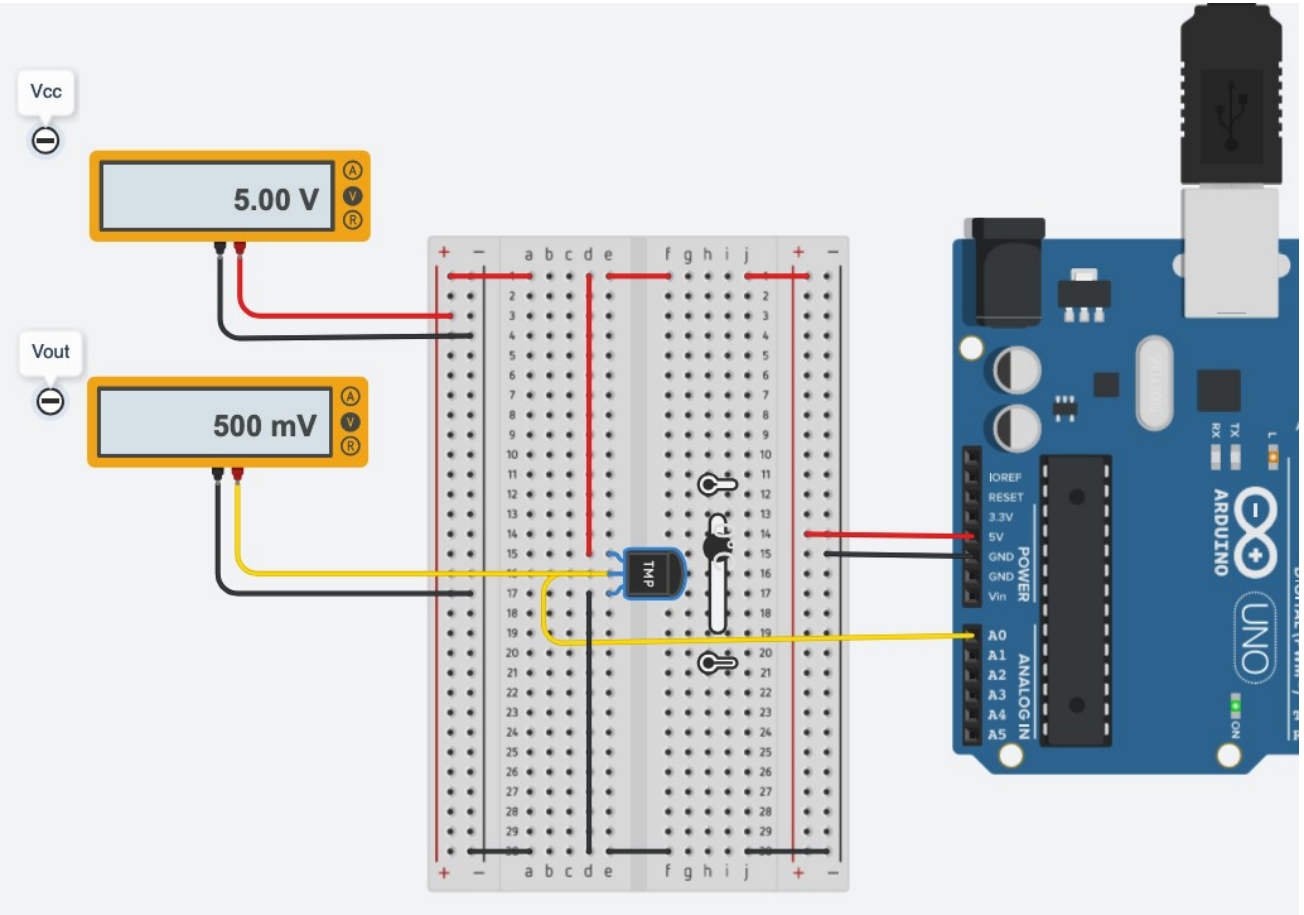
Lab1: Task 1. Introducing a mixed signal circuit design.



In this screenshot we show the temperature sensor set at 25ºC. The input voltmeter show 5V as expected, and Vout shows 0V.

In the next task, we are going to connect the voltmeter and the temperature sensors to get the output voltage.

Lab1: Task 1. Introducing a mixed signal circuit design.



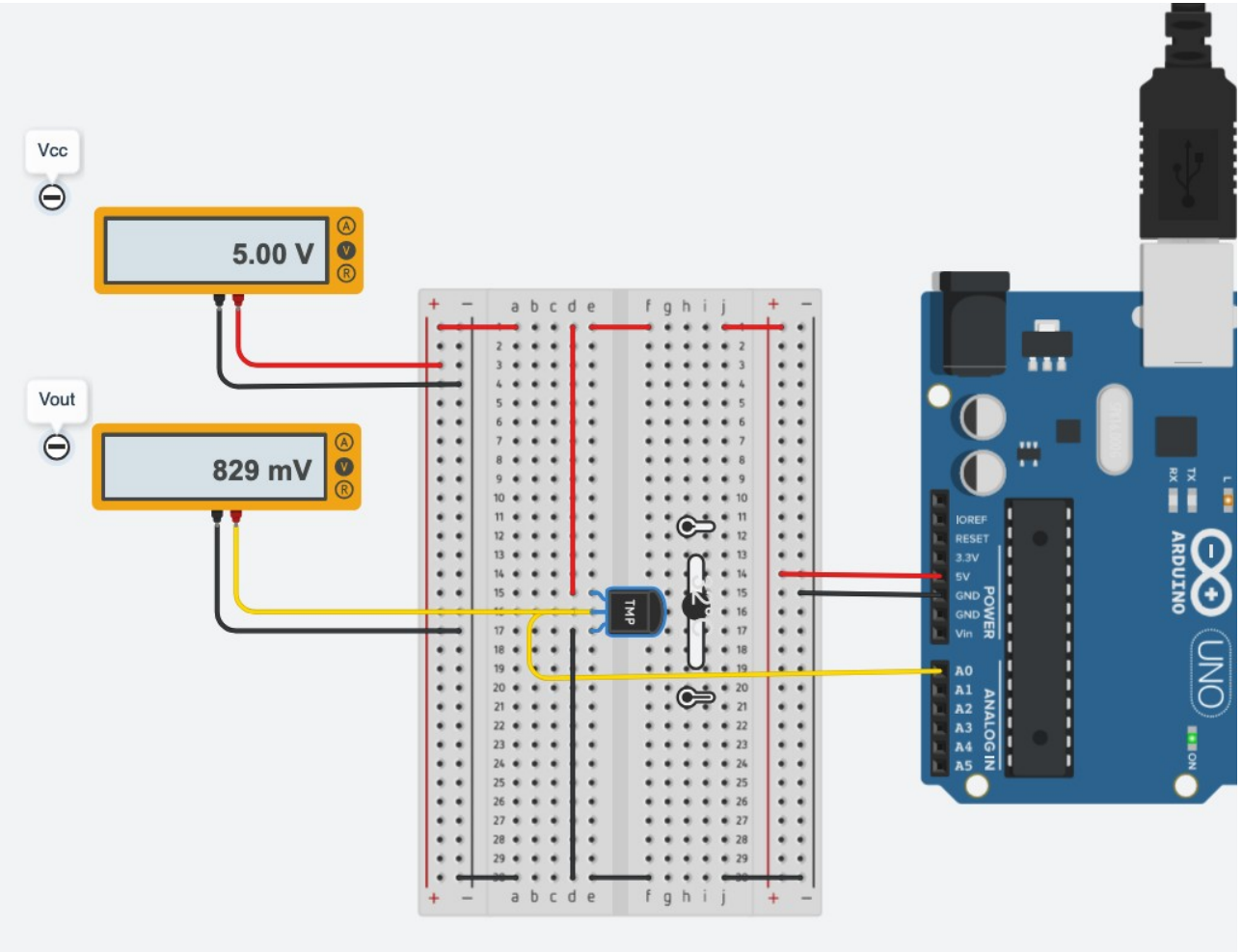
In this screenshot the temperature sensor for the minimum temperature configured at 0ºC, the Vout shows 500mV.

Lab1: Task 1. Introducing a mixed signal circuit design.

	T (ºC)	Vin	Vout
Minimum Temp value	0ºC	5V	0V
T0	25ºC	5V	0V
T1	25ºC	5V	0V
Maximum Temp value	80ºC	5V	0V

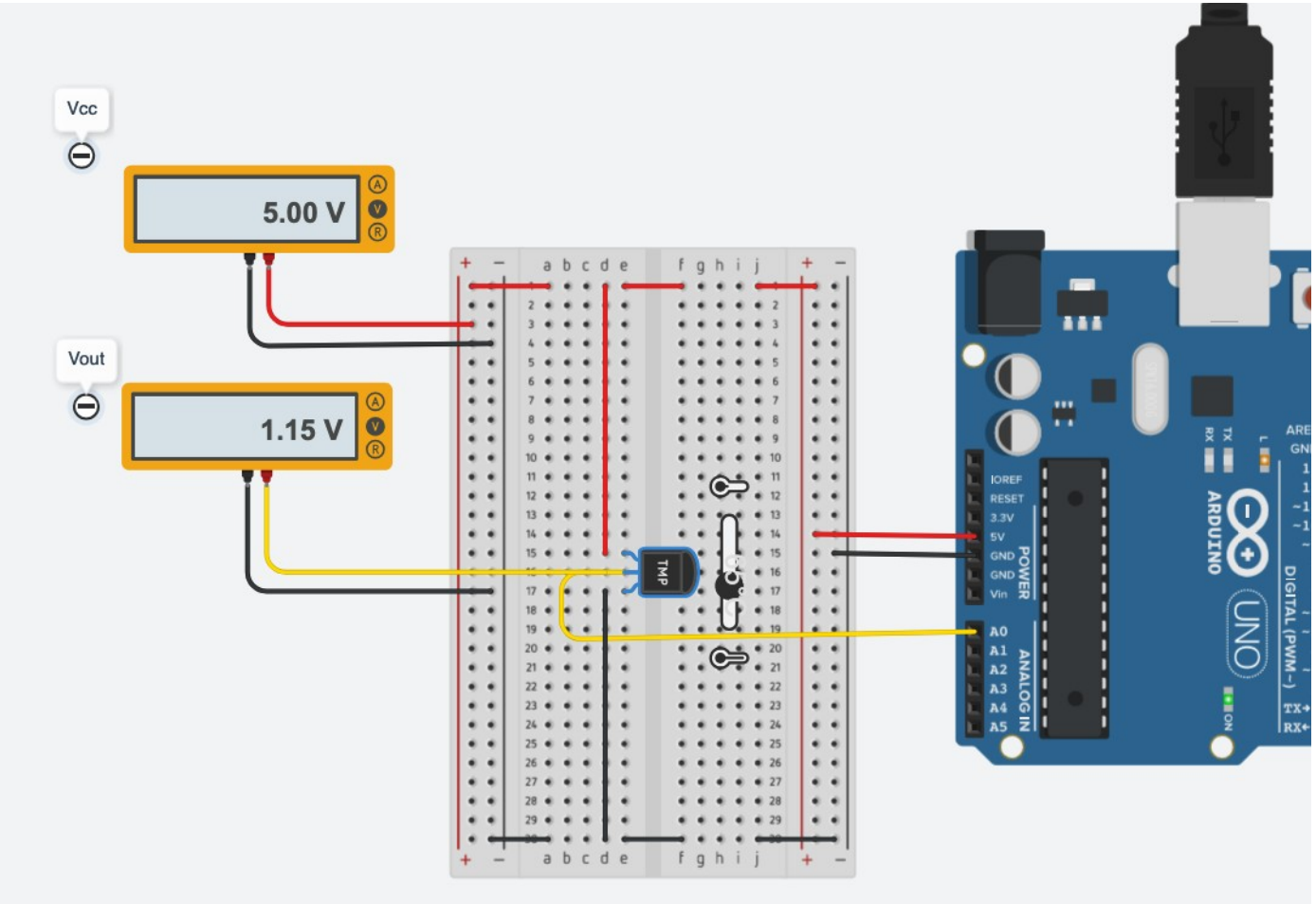
Table 1.1 - The table shows the values with analog port simulated. That's mean we show only the input value in the first multimeter. In the second one is *null* value because we don't convert the analog value to digital from the temperature sensor.

Lab1: Task 1. Introducing a mixed signal circuit design.



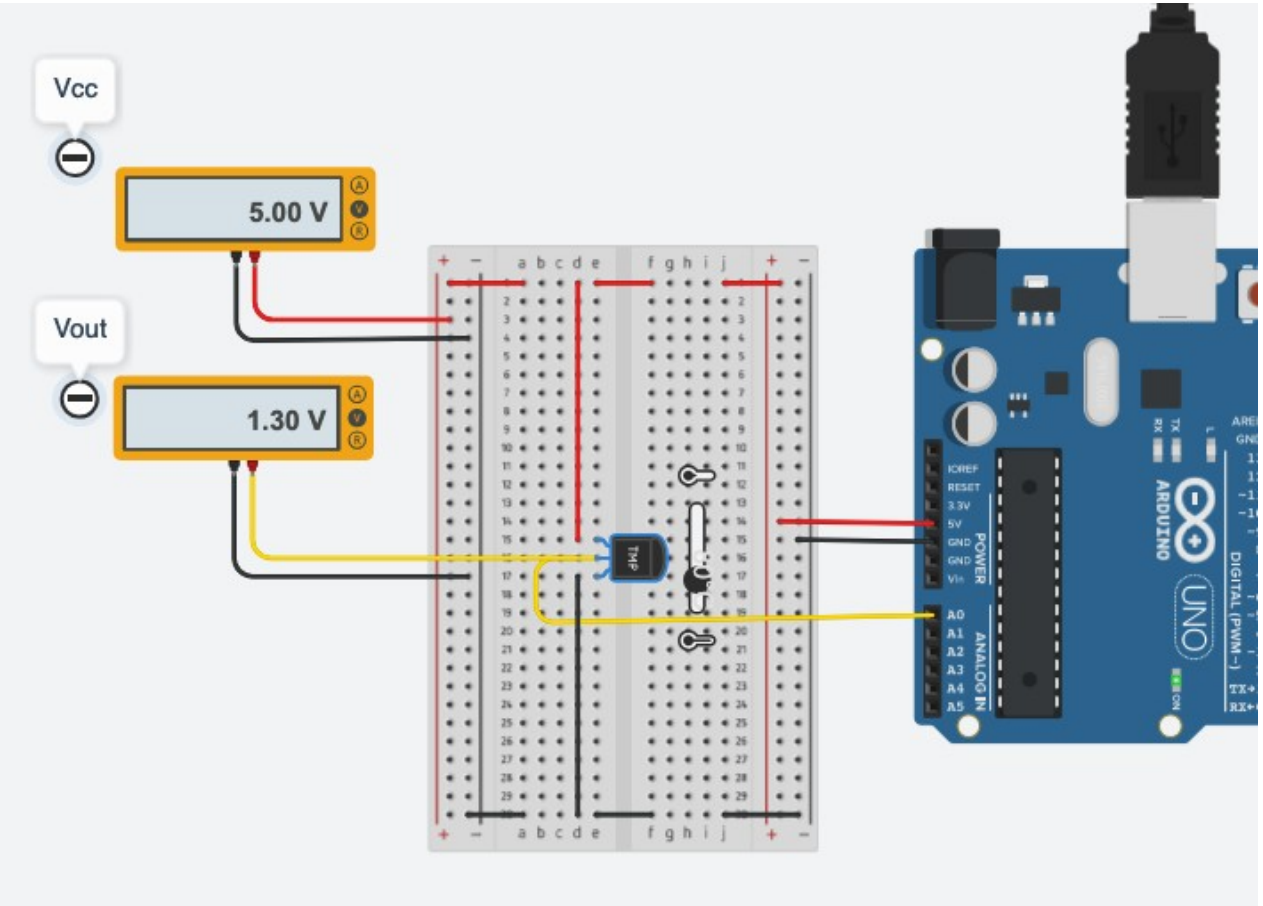
In this screenshot we see the temperature sensor set at 32ºC. The input voltmeter show 5V as expected, and Vout shows ~829mV.

Lab1: Task 1. Introducing a mixed signal circuit design.



In this screenshot the temperature sensor has been configured at 65ºC, and voltmeter out shows the corresponding voltage, 1.15V

Lab1: Task 1. Introducing a mixed signal circuit design.



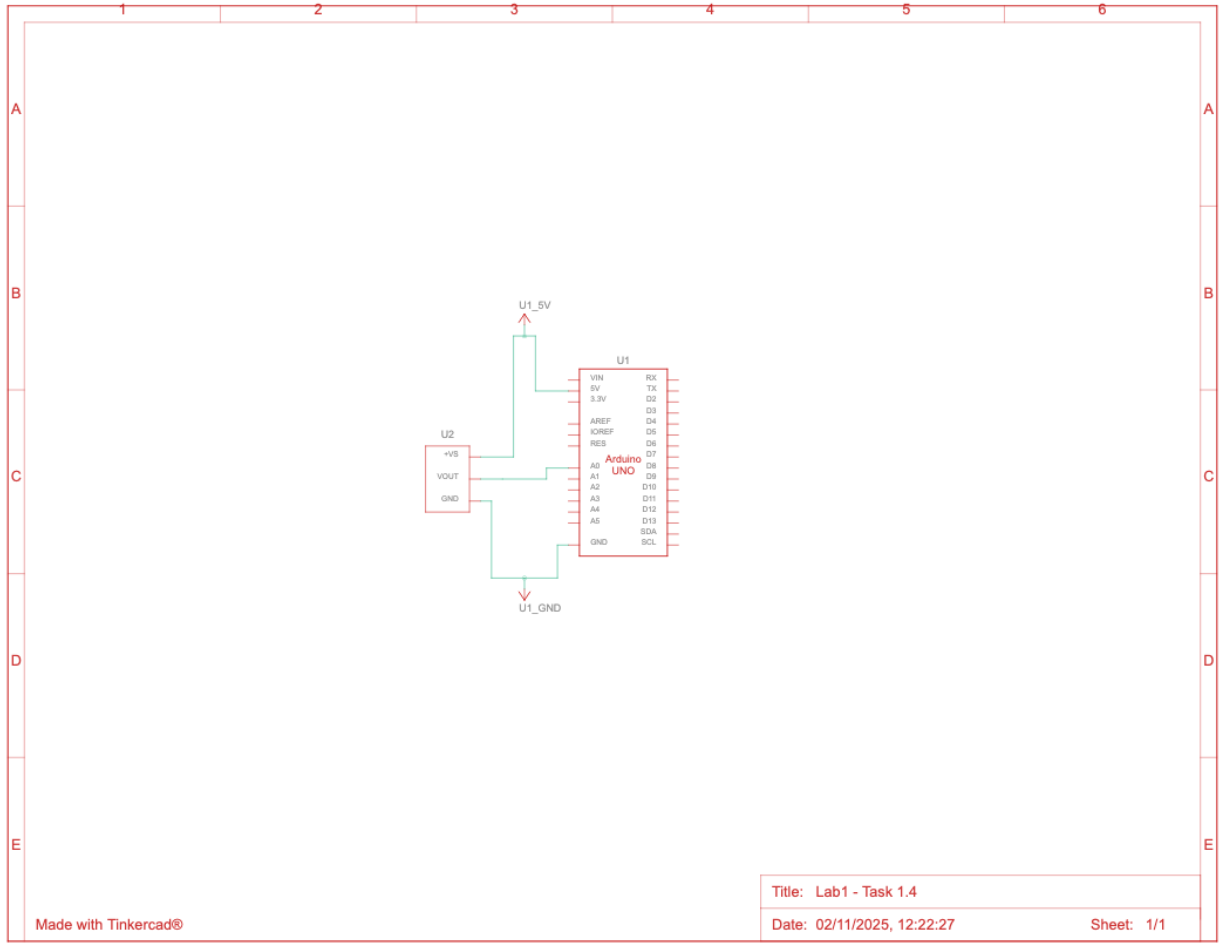
In this screenshot the temperature sensor has been configured at 80ºC, and voltmeter out shows the corresponding voltage, 1.30V

Lab1: Task 1. Introducing a mixed signal circuit design.

	T (ºC)	Vin	Vout
Minimum Temp value	0ºC	5V	500mV
T0	32ºC	5V	829mV
T1	65ºC	5V	1.15V
Maximum Temp value	80ºC	5V	1.3V

Table 1.4 - The table shows the values with analog port wired. In the first voltmeter always shows the power supply voltage, 5V. In the second one, the values depends of the temperature sensors, which convert the analog value to digital.

Lab1: Task 1. Introducing a mixed signal circuit design.



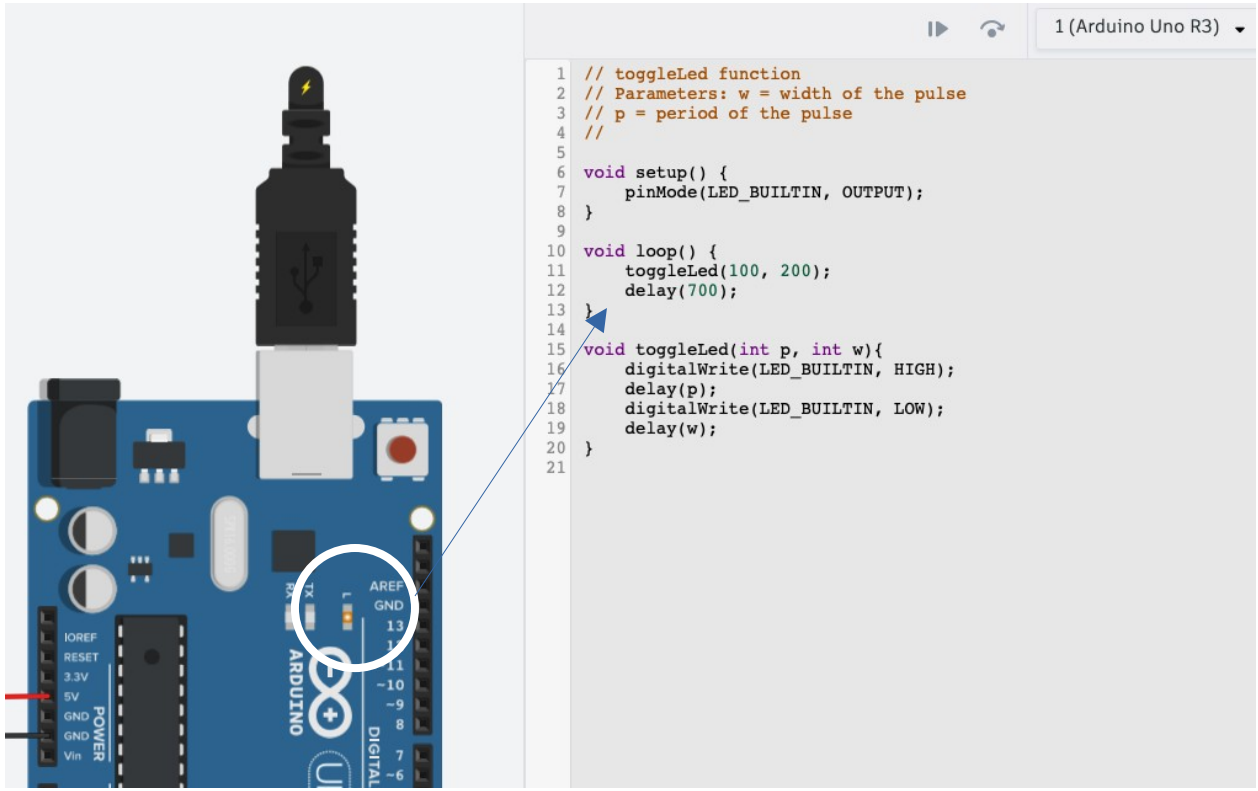
In the last screenshot of task 1.4, we will show the schematic view of the circuit.

So, me move forward to next step 2. We are going to remove breadboard and connect the temperature sensor by A0 analog port using coding to shows the output results by text.

Laboratory 1 Report

- Task 1 – Schematics of a mixed signal circuit.
- **Task 2 – Programming the Arduino board.**
- Task 3 – Introducing the LCD display.
- Task 4 – Conditioning circuit.
- Task 5 – Results.

Lab1: Task 2. Programming the Arduino board



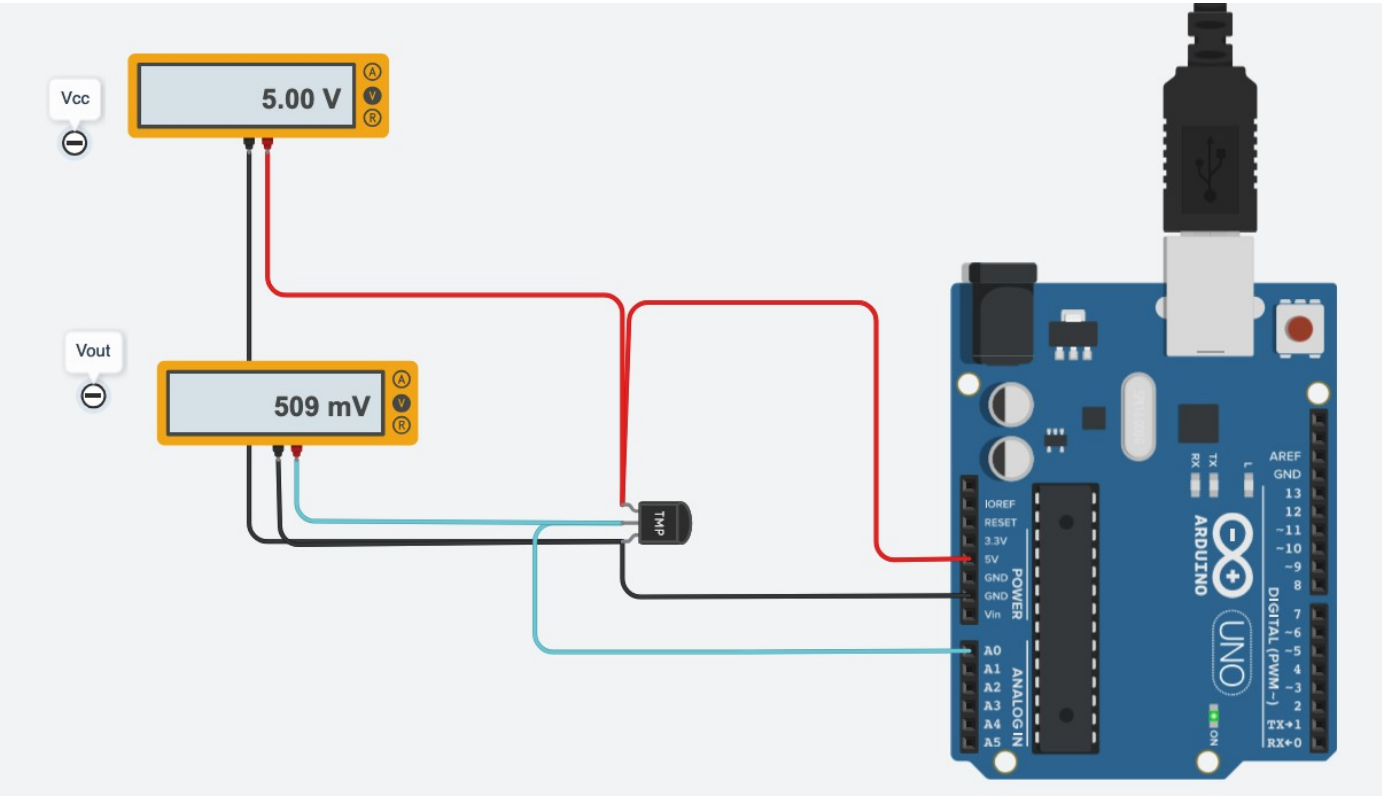
The following code used, we code on-board Led blink by specific C programming language within Arduino. In the screenshot we mark it with white circle as result.

Anyway, only for the first **task 2.1**, we set 100ms of lightening and 200ms off.

Basically, in the function *setup* we set how many milliseconds the led blinks and the delay duration, set to 700ms.

The function *loop*, read the main function **toggleLed**, with two arguments, *p* (pulse) and *w* (width) and assign the delay per each.

Lab1: Task 2. Programming the Arduino board

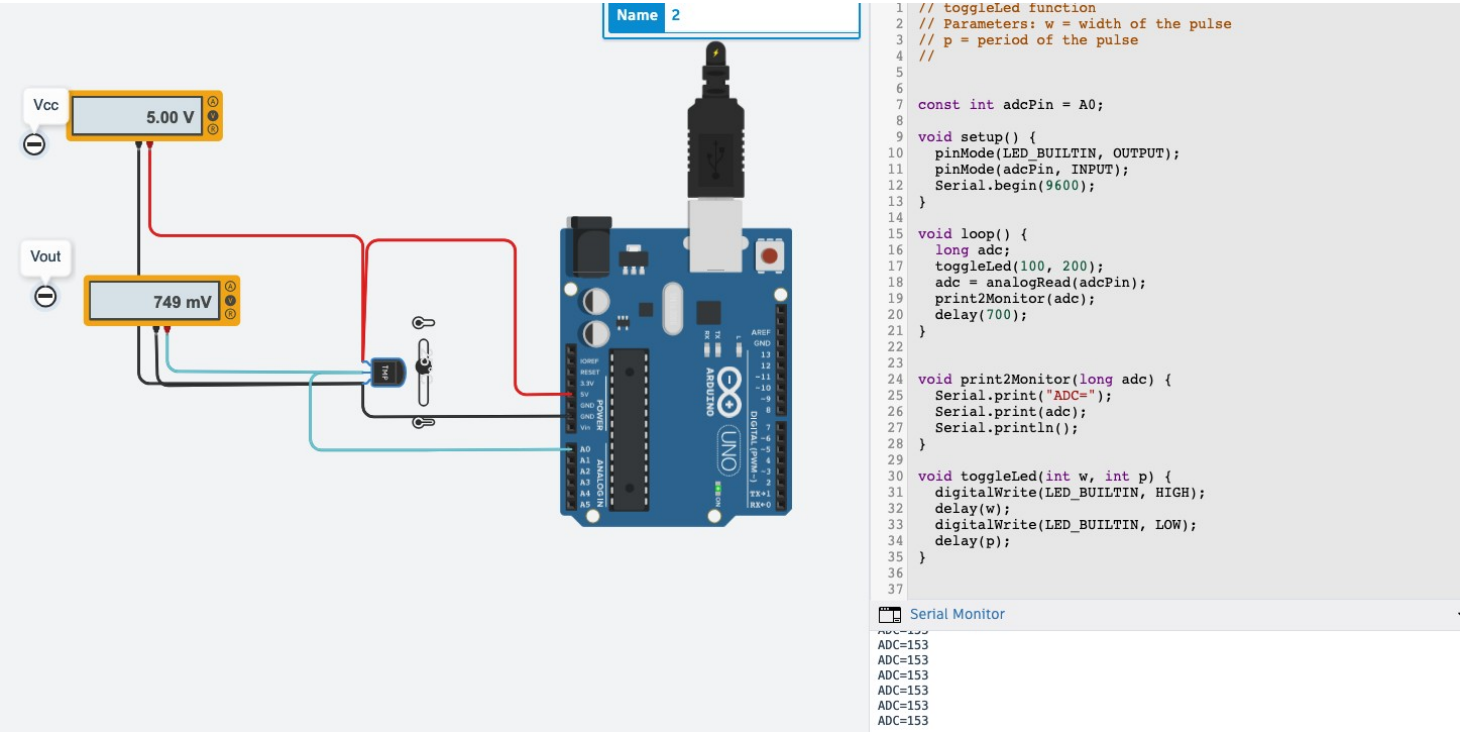


In the **task 2.1** we show the best approach to programming on Arduino UNO Board, using the code section and interact with the all physical components. As group E, we have temperature sensor with a range 0ºC to 80ºC.

In this screenshot, we are able to show the temperature sensor connected via analog port, A0. In the first voltmeter, we still see the input voltage, 5V. In the second one, we show the output voltage of temperature sensor, the Vout.

As the first task, we are going to show some basic function, such as how the on-board led blink works.

Lab1: Task 2. Programming the Arduino board

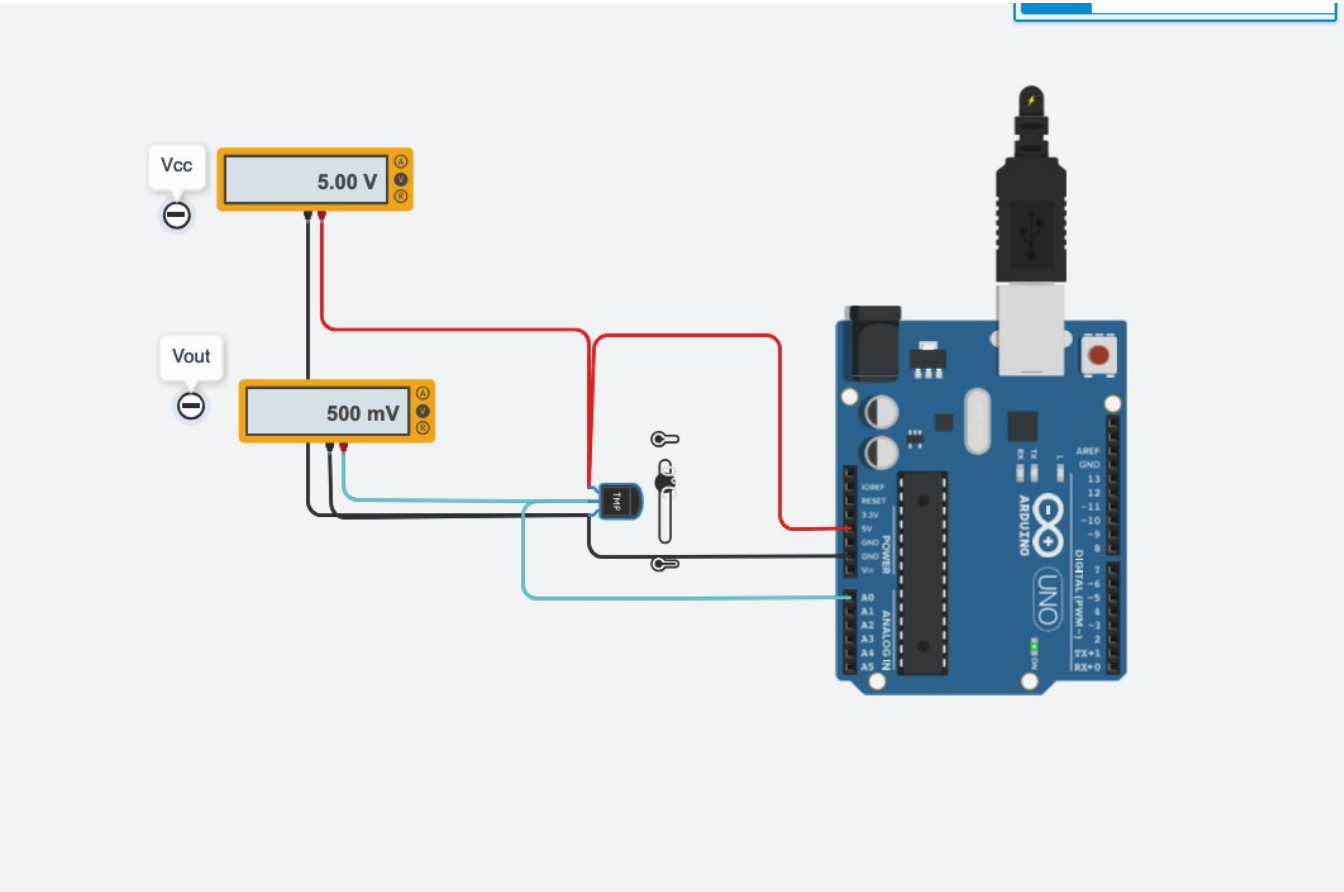


In the **task 2.2**, we add some functions more to show the analog to digital conversion in the serial monitor (the output text below the code section). The requirement is interact directly with the temperature sensor, using the slicer with the range 0ºC to 80ºC.

After set the input variable, the constant **adcPin** connected by A0 analog port, we add more functions in the main code. The first to convert the analog signal to digital. And the second function, to show the output text on the **Serial Monitor**.

So, the variable *adcPin* get from the port A0 the analog value from the temperature sensor, and the it shows in the serial monitor the digital value. It's expressed as a number between 0 and 1023.

Lab1: Task 2. Programming the Arduino board



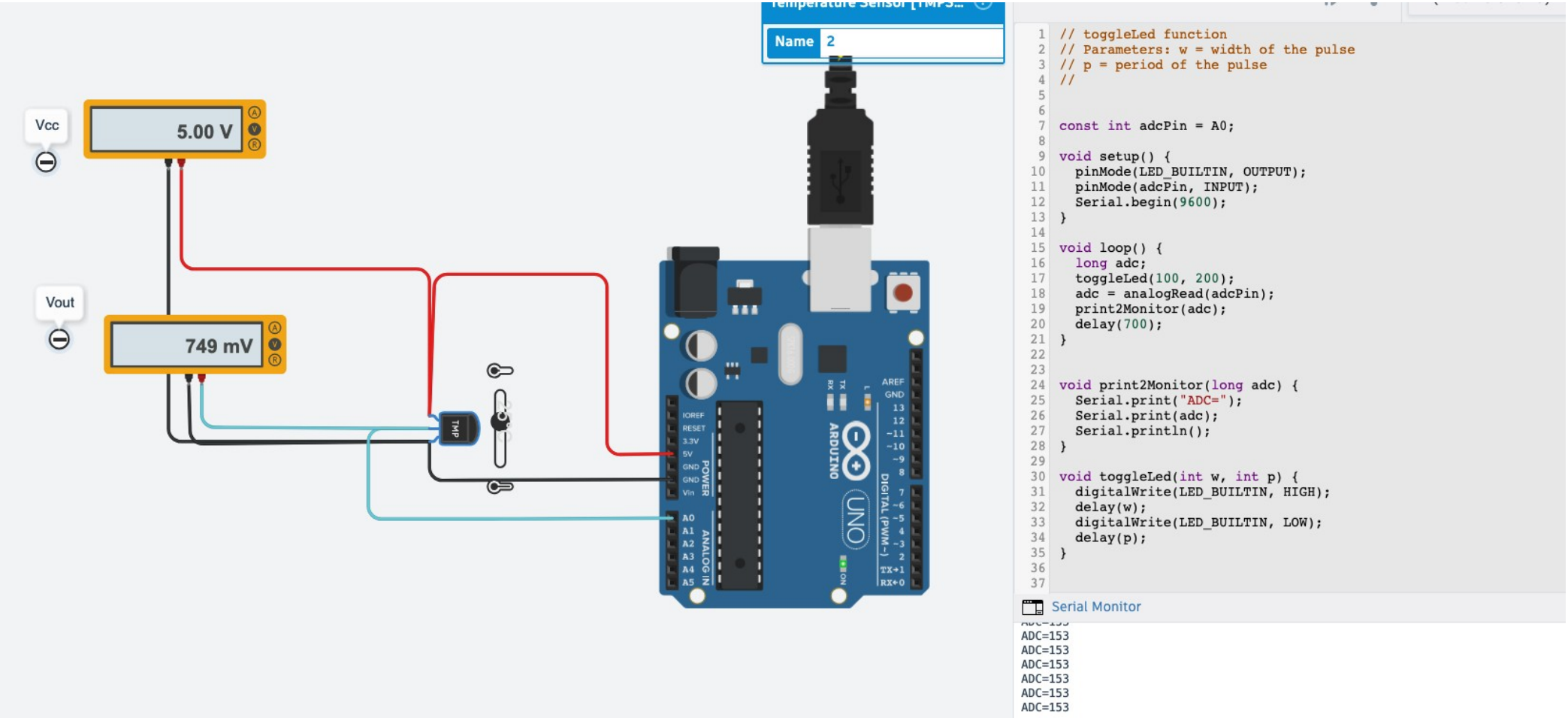

```
// p = period of the pulse
//


const int adcPin = A0;

void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
  pinMode(adcPin, INPUT);
  Serial.begin(9600);
}

void loop() {
  long adc;
  toggleLed(100, 200);
  adc = analogRead(adcPin);
  print2Monitor(adc);
  delay(700);
}


void print2Monitor(long adc) {
  Serial.print("ADC=");
  Serial.print(adc);
  Serial.println();
}

void toggleLed(int w, int p) {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(w);
  digitalWrite(LED_BUILTIN, LOW);
  delay(p);
}
```

Serial Monitor

```
ADC=102
ADC=102
ADC=102
ADC=102
ADC=102
ADC=102
```

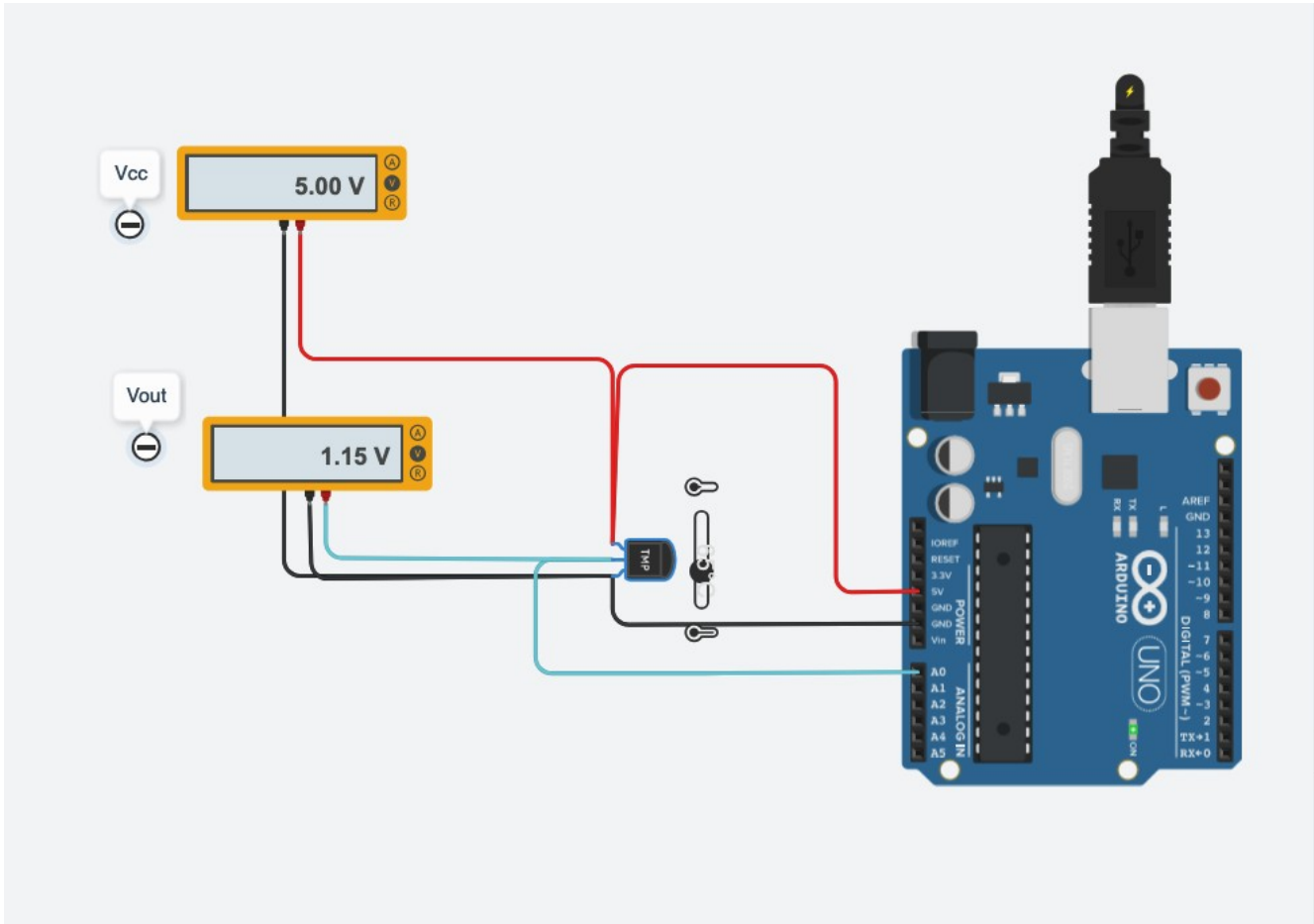
In this screenshot we show the temperature sensor set to minimum ~0ºC; the output voltage ~500mV and in the bottom-right the analog value converted to digital, ADC=~102.

Lab1: Task 2. Programming the Arduino board



In this screenshot we show the temperature sensor set to 25ºC; the output voltage ~749mV and in the bottom-right the analog value converted to digital ADC=153.

Lab1: Task 2. Programming the Arduino board



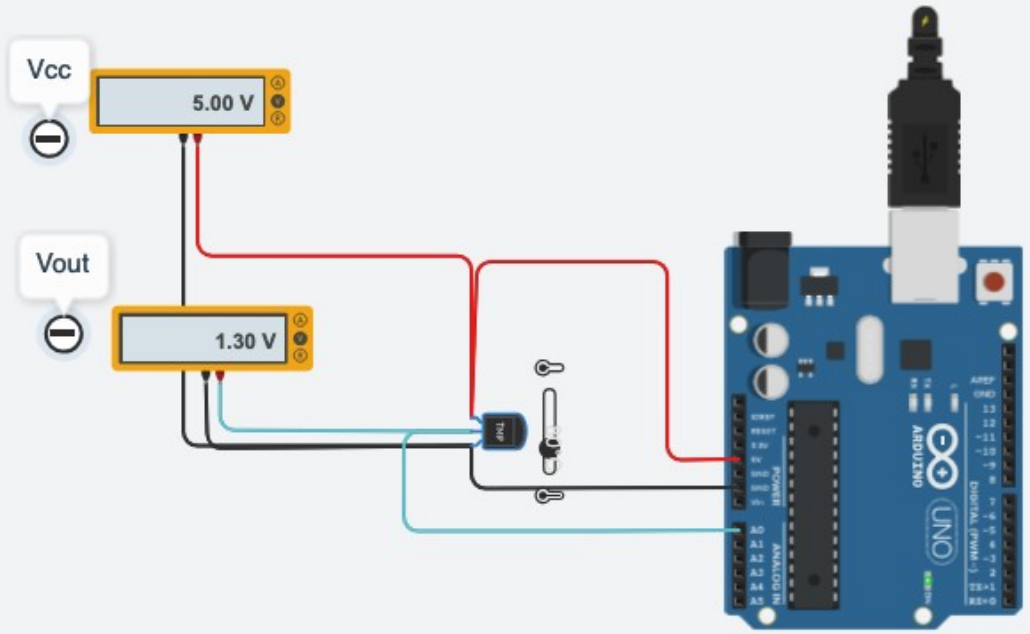

```
//


const int adcPin = A0;

void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
  pinMode(adcPin, INPUT);
  Serial.begin(9600);
}

void loop() {
  long adc;
  toggleLed(100, 200);
  adc = analogRead(adcPin);
  print2Monitor(adc);
  delay(700);
}


void print2Monitor(long adc) {
  Serial.print("ADC=");
  Serial.print(adc);
  Serial.println();
}

void toggleLed(int w, int p) {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(w);
  digitalWrite(LED_BUILTIN, LOW);
  delay(p);
}
```

Serial Monitor

```
ADC=235
ADC=235
ADC=235
ADC=235
ADC=235
ADC=235
ADC=235
```

In this screenshot we show the temperature sensor set to 65ºC; the output voltage ~1.15V and in the bottom-right the analog value converted to digital ADC=235.

Lab1: Task 2. Programming the Arduino board




```
}

void loop() {
  long adc;
  toggleLed(100, 200);
  adc = analogRead(adcPin);
  print2Monitor(adc);
  delay(700);
}


void print2Monitor(long adc) {
  Serial.print("ADC=");
  Serial.print(adc);
  Serial.println();
}

void toggleLed(int w, int p) {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(w);
  digitalWrite(LED_BUILTIN, LOW);
  delay(p);
}
```

Serial Monitor

```
ADC=266
ADC=266
ADC=266
ADC=266
ADC=266
ADC=266
```

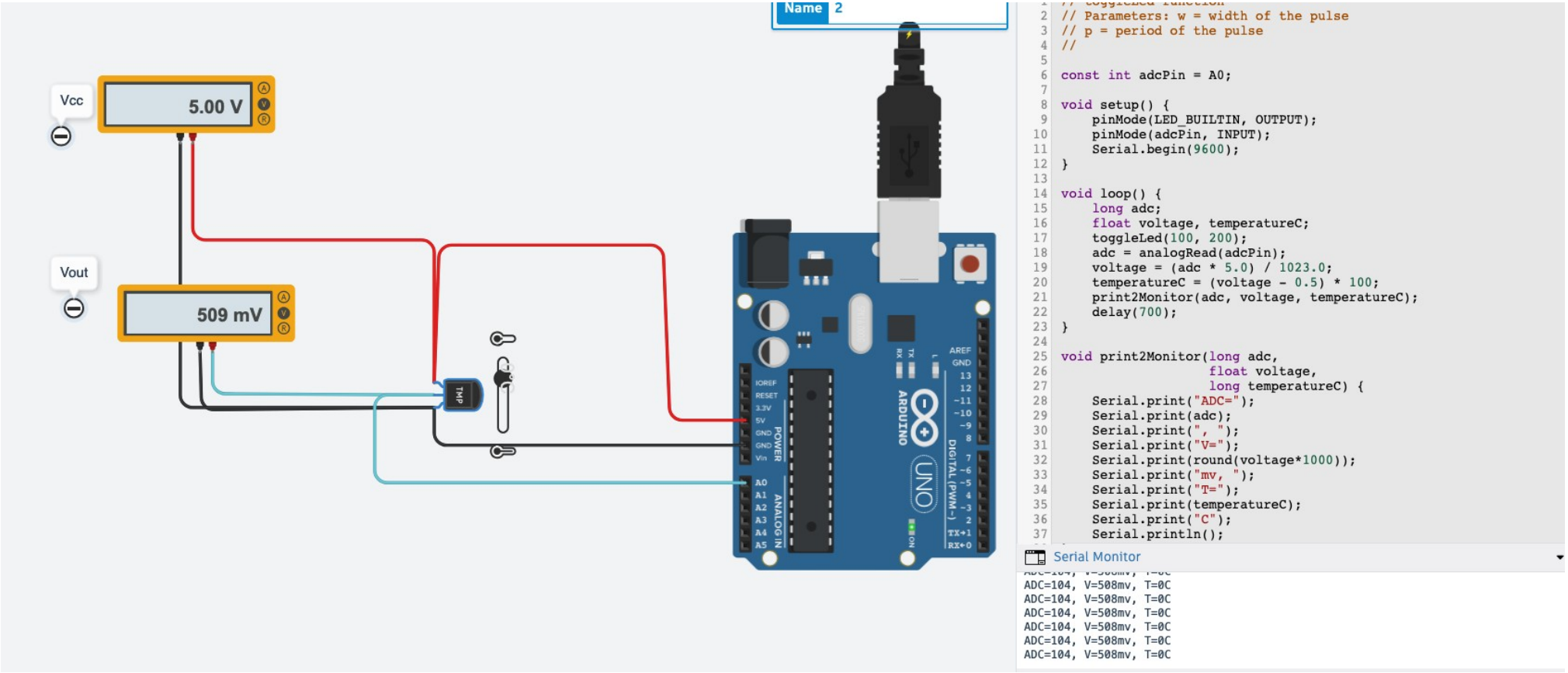
In this screenshot we show the temperature sensor set to maximum 80ºC; the output voltage ~1.30V and in the bottom-right the analog value converted to digital ADC=266.

Lab1: Task 2. Programming the Arduino board

	T (ºC)	Vin	Vout	ADC value
Minimum Temp value	0ºC	5V	~500mV	~102
T0	25ºC	5V	~749mv	~153
T1	65ºC	5V	~1.15V	~235
Maximum Temp value	80ºC	5V	~1.30V	~266

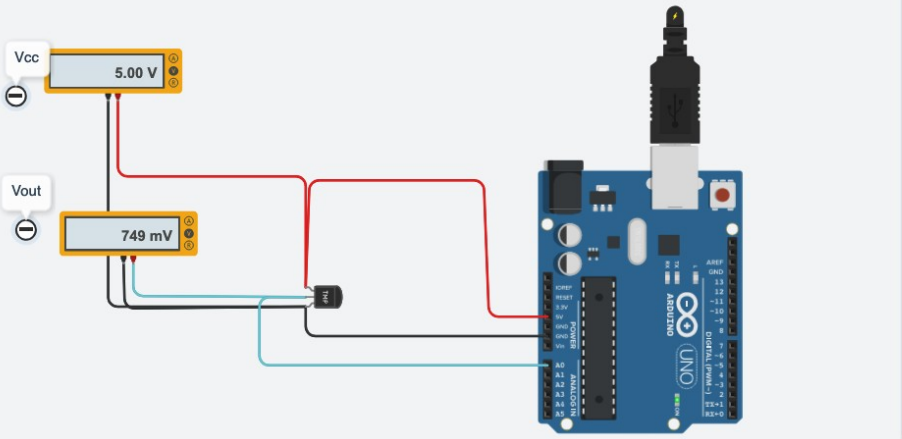
Table III - The table shows the values with analog port connected with the temperature sensor.
That's mean we are able to show the output voltage and the original analog value in the Serial Monitor.
It's important to remind that the output values are the approximation, due the oscillations of the input voltage, isn't 5V precisely and the temperature sensor doesn't set the decimal part.

Lab1: Task 2. Programming the Arduino board



In this screenshot we show the output values in the Serial Monitor. In order, there are the analog value converted to digital ADC=104; the output voltage ~509mV and the temperature sensor set to minimum requested at 0ºC.

Lab1: Task 2. Programming the Arduino board




```
// Parameters: w = width of the pulse
// p = period of the pulse
//

const int adcPin = A0;

void setup() {
    pinMode(LED_BUILTIN, OUTPUT);
    pinMode(adcPin, INPUT);
    Serial.begin(9600);
}

void loop() {
    long adc;
    float voltage, temperatureC;
    toggleLed(100, 200);
    adc = analogRead(adcPin);
    voltage = (adc * 5.0) / 1023.0;
    temperatureC = (voltage - 0.5) * 100;
    print2Monitor(adc, voltage, temperatureC);
    delay(700);
}

void print2Monitor(long adc,
                   float voltage,
                   long temperatureC) {
    Serial.print("ADC=");
    Serial.print(adc);
    Serial.print(", ");
    Serial.print("V=");
    Serial.print(round(voltage*1000));
    Serial.print("mv, ");
    Serial.print("T=");
    Serial.print(temperatureC);
    Serial.print("C");
    Serial.println();
```

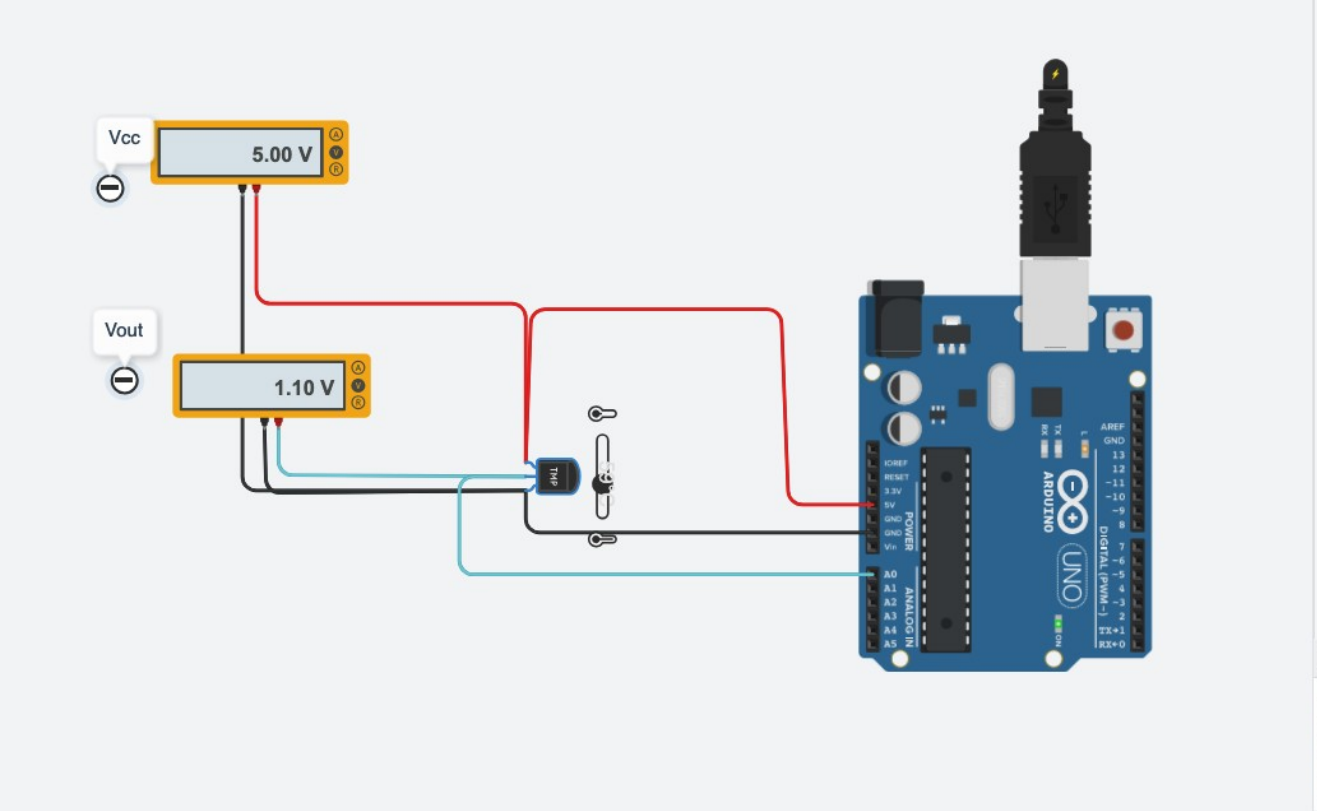
Serial Monitor

```
ADC=153, V=748mv, T=24C
ADC=153, V=748mv, T=24C
ADC=153, V=748mv, T=24C
ADC=153, V=748mv, T=24C
ADC=153, V=748mv, T=24C
ADC=153, V=748mv, T=24C
```

In the **task 2.3**, we improve the initial code with one more function to convert the analog value to digital. Additionally, we are able to show the corresponding converted value to mV as the output on the Serial Monitor.

In the screenshot we show the main code used, specially to convert the values for *voltage* and *temperature.*

Lab1: Task 2. Programming the Arduino board



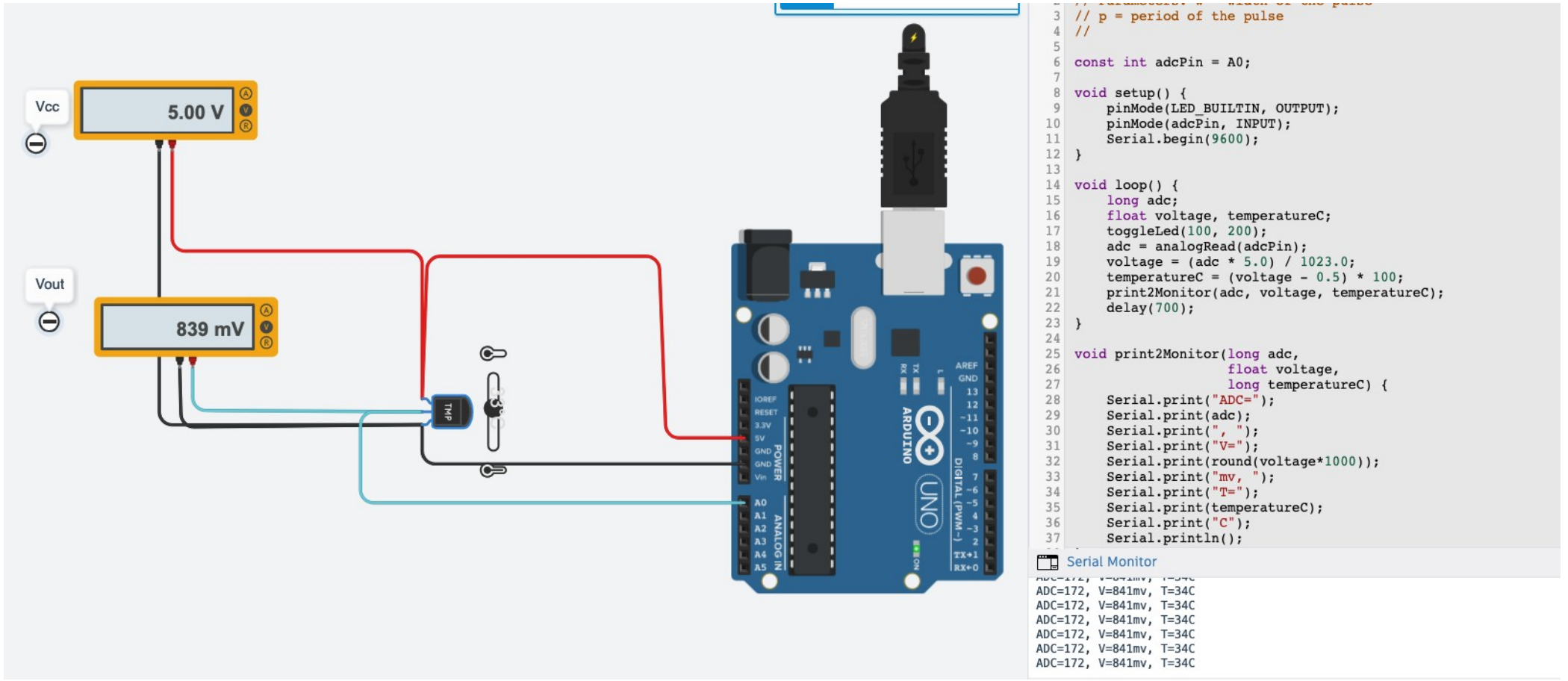

```
void setup() {
    pinMode(LED_BUILTIN, OUTPUT);
    pinMode(adcPin, INPUT);
    Serial.begin(9600);
}

void loop() {
    long adc;
    float voltage, temperatureC;
    toggleLed(100, 200);
    adc = analogRead(adcPin);
    voltage = (adc * 5.0) / 1023.0;
    temperatureC = (voltage - 0.5) * 100;
    print2Monitor(adc, voltage, temperatureC);
    delay(700);
}

void print2Monitor(long adc,
                   float voltage,
                   long temperatureC) {
    Serial.print("ADC=");
    Serial.print(adc);
    Serial.print(", ");
    Serial.print("V=");
    Serial.print(round(voltage*1000));
    Serial.print("mv, ");
    Serial.print("T=");
    Serial.print(temperatureC);
    Serial.print("C");
    Serial.println();
```

Serial Monitor

```
ADC=225, V=1100mv, T=59C
ADC=225, V=1100mv, T=59C
ADC=225, V=1100mv, T=59C
ADC=225, V=1100mv, T=59C
ADC=225, V=1100mv, T=59C
ADC=225, V=1100mv, T=59C
ADC=225, V=1100mv, T=59C
```

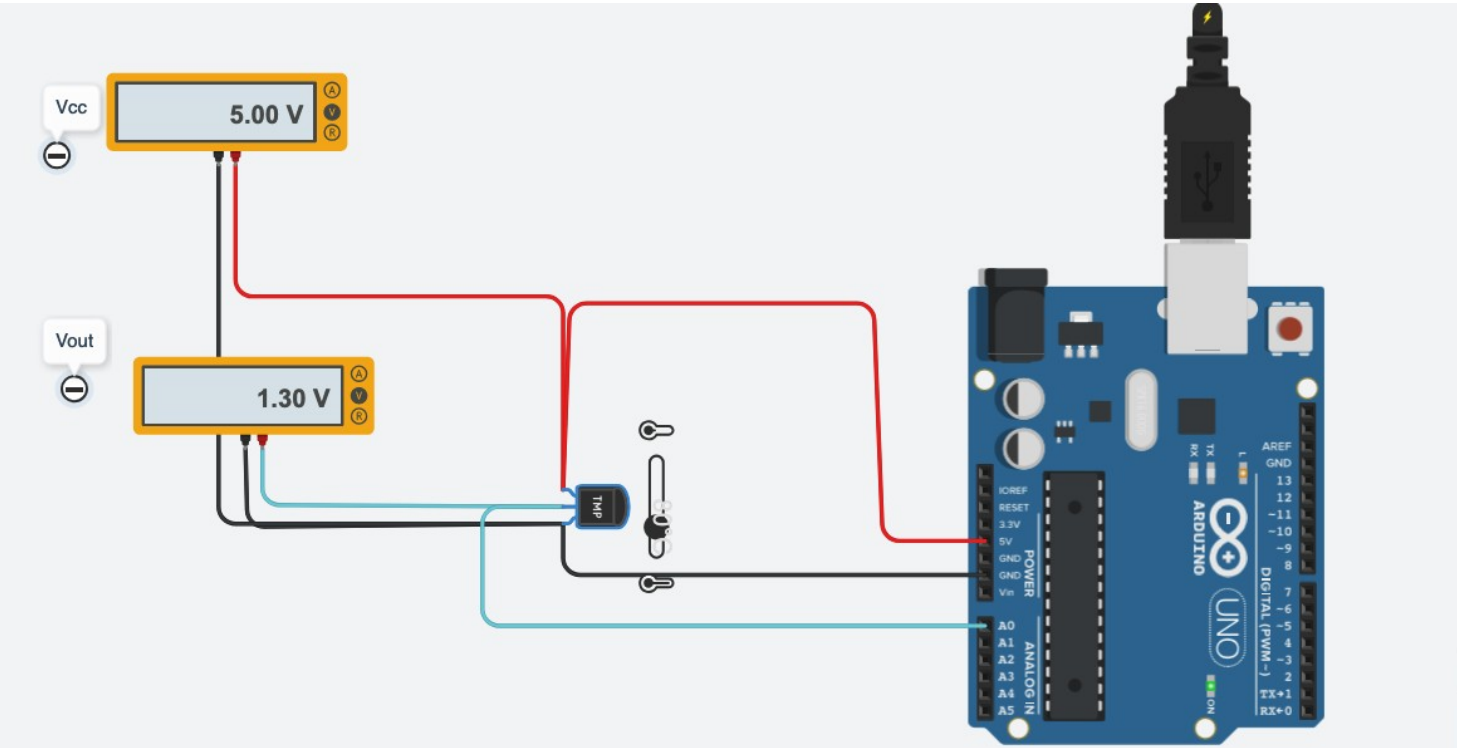
In this screenshot we show the output values in the Serial Monitor. In order, there are the analog value converted to digital ADC=225; the output voltage ~1100mV and the temperature sensor set to T2 requested at 59ºC.

Lab1: Task 2. Programming the Arduino board



In this screenshot we show the output values in the Serial Monitor. In order, there are the analog value converted to digital ADC=172; the output voltage ~839mV and the temperature sensor set to T1 requested at 34ºC.

Lab1: Task 2. Programming the Arduino board




```
const int adcPin = A0;

void setup() {
    pinMode(LED_BUILTIN, OUTPUT);
    pinMode(adcPin, INPUT);
    Serial.begin(9600);
}

void loop() {
    long adc;
    float voltage, temperatureC;
    toggleLed(100, 200);
    adc = analogRead(adcPin);
    voltage = (adc * 5.0) / 1023.0;
    temperatureC = (voltage - 0.5) * 100;
    print2Monitor(adc, voltage, temperatureC);
    delay(700);
}

void print2Monitor(long adc,
                   float voltage,
                   long temperatureC) {
    Serial.print("ADC=");
    Serial.print(adc);
    Serial.print(", ");
    Serial.print("V=");
    Serial.print(round(voltage*1000));
    Serial.print("mv, ");
    Serial.print("T=");
    Serial.print(temperatureC);
    Serial.print("C");
    Serial.println();
```

Serial Monitor

```
ADC=266, V=1300mv, T=80C
ADC=266, V=1300mv, T=80C
ADC=266, V=1300mv, T=80C
ADC=266, V=1300mv, T=80C
ADC=266, V=1300mv, T=80C
ADC=266, V=1300mv, T=80C
ADC=266, V=1300mv, T=80C
```

In this screenshot we show the output values in the Serial Monitor. In order, there are the analog value converted to digital ADC=266; the output voltage ~1300mV and the temperature sensor set to maximum requested at 80ºC.

Lab1: Task 2. Programming the Arduino board

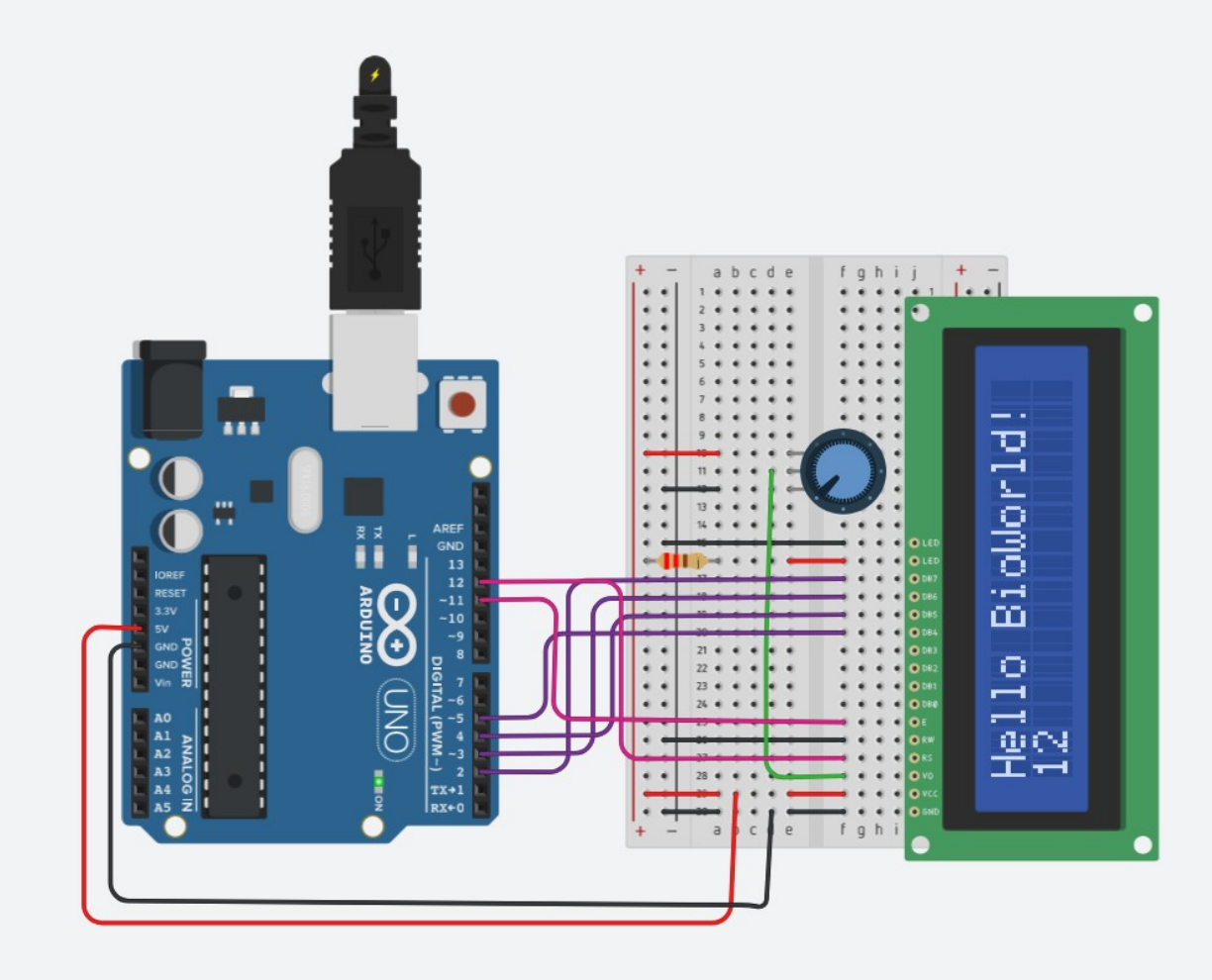
	T (ºC)	Vout (V)	ADC steps	Vinadc (mV)	Tarduino
Minimum Temp. value	0ºC	0V	0	0mV	0.0
T1	34ºC	2.13V	434	2124mV	33.9
T2	59ºC	3.69V	754	3684mV	58.9
Maximum Temp. value	80ºC	5.0V	1023	5000mV	80.0

Table IV – It's a final table if we set the power voltage with a minimum temperature range. 0ºC = 0V and 80ºC = 5V. The T1 and T2 are calculated to return the corresponding values for ADC steps and Vinadc.

Laboratory 1 Report

- Task 1 – Schematics of a mixed signal circuit.
- Task 2 – Programming the Arduino board.
- **Task 3 – Introducing the LCD display.**
- Task 4 – Conditioning circuit.
- Task 5 – Results.

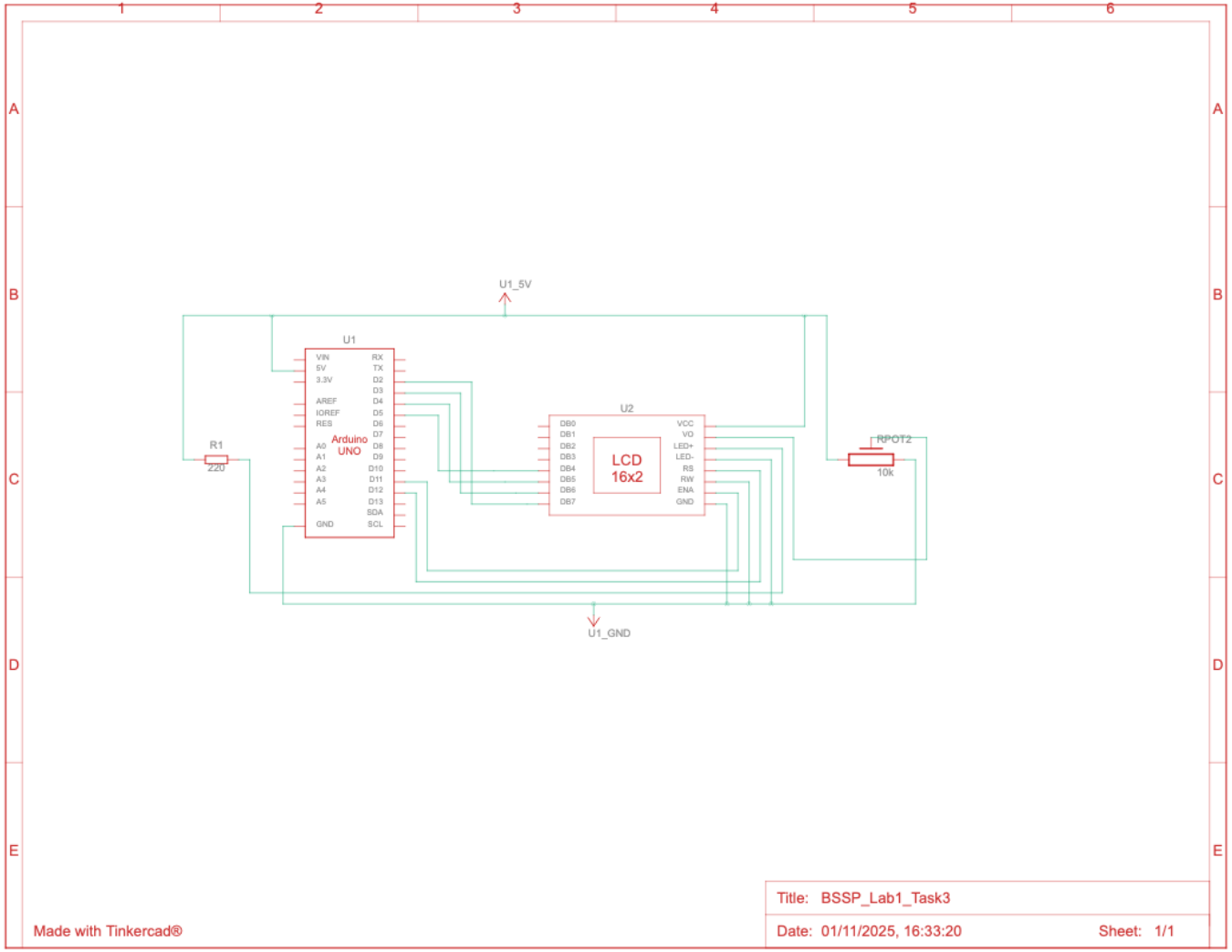
Lab1: Task 3. Introducing the LCD display.



In the **task 3.1**, we design a circuit to connect a potentiometer and a 16x2 LCD display. The following components are used to achieve it:
- 1x Arduino UNO board;
- 1x breadboard small;
- 1x 16x2 LCD display;
- 1x potentiometer configured at 10kΩ;
- 1x resistor with value at 220Ω;
- red wire to supply 5V; black wire at ground; pink and purple wires to connect the LCD display.

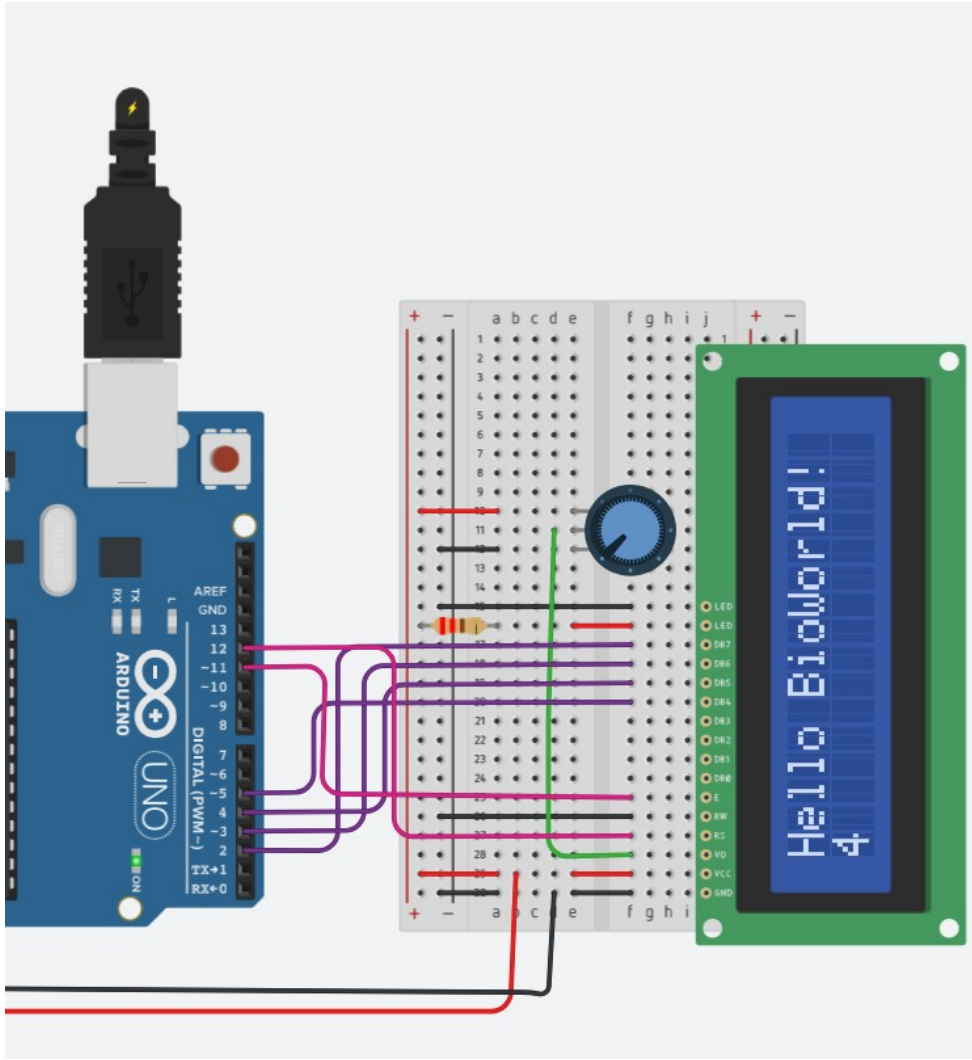
In the next two slides we are going to show the schematic view to understand how connect each pin between Arduino and LCD display. And the others, show the code used, import LCD display function from Arduino library.

Lab1: Task 3. Introducing the LCD display.



The screenshot shows the schematic view of Arduino UNO, LCD display, resistor and the potentiometer connected all together.

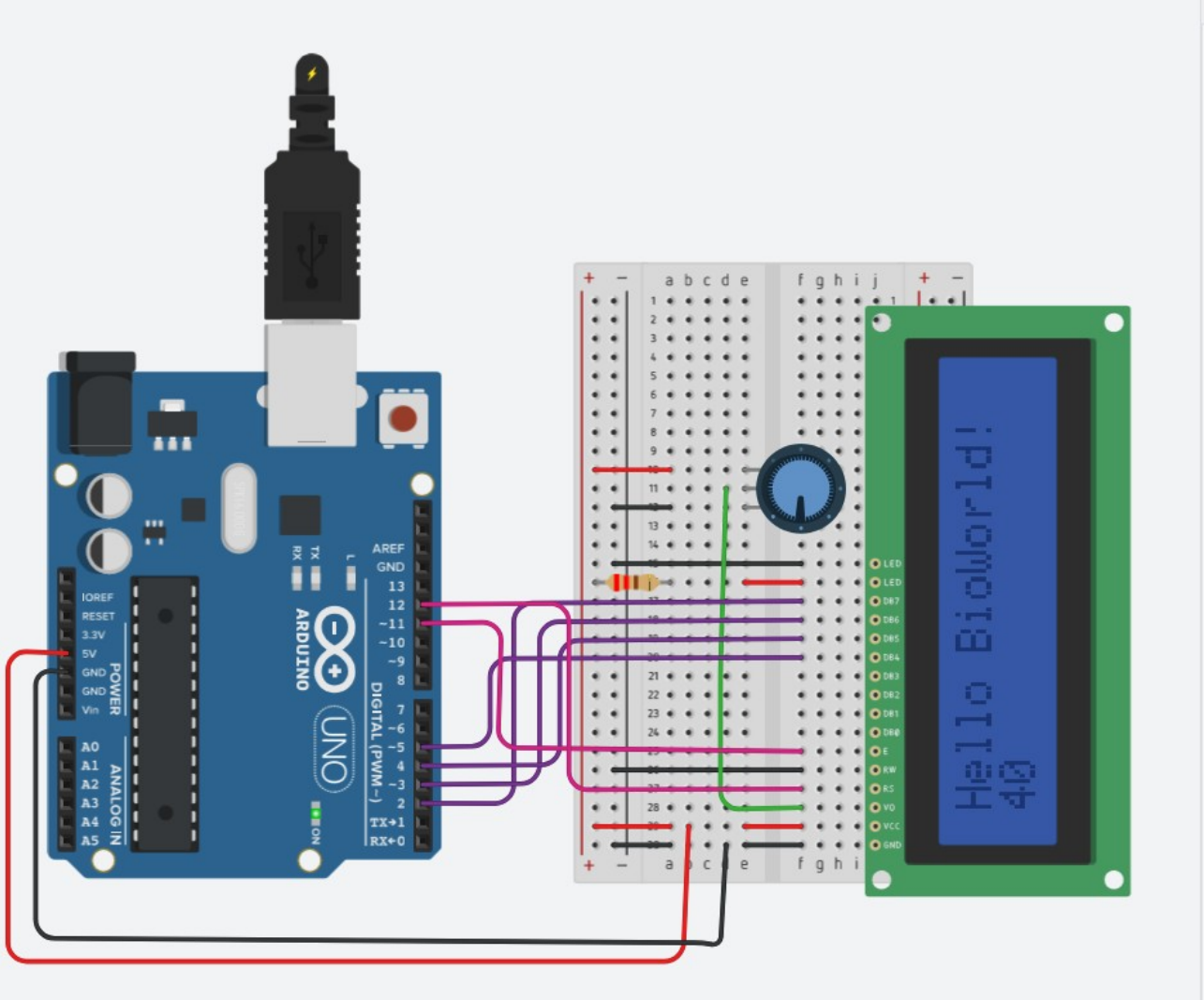
Lab1: Task 3. Introducing the LCD display.



1 (Arduino Uno R3)

```
 * LCD D6 pin to digital pin 3
 * LCD D7 pin to digital pin 2
 * LCD R/W pin to ground
 * LCD VSS pin to ground
 * LCD VCC pin to 5V
 * 10K resistor:
 * ends to +5V and ground
 * wiper to LCD VO pin (pin 3)

 Library originally added 18 Apr 2008
 by David A. Mellis
 library modified 5 Jul 2009
 by Limor Fried (http://www.ladyada.net)
 example added 9 Jul 2009
 by Tom Igoe
 modified 22 Nov 2010
 by Tom Igoe
 modified 7 Nov 2016
 by Arturo Guadalupi

 This example code is in the public domain.

 https://docs.arduino.cc/learn/electronics/lcd-displays

*/

// include the library code:
#include <LiquidCrystal.h>

// initialize the library by associating any needed LCD interface
// with the arduino pin number it is connected to
const int rs = 12, en = 11, d4 = 5, d5 = 4, d6 = 3, d7 = 2;
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);

void setup() {
  // set up the LCD's number of columns and rows:
  lcd.begin(16, 2);
  // Print a message to the LCD.
  lcd.print("Hello BioWorld!");
}

void loop() {
  // set the cursor to column 0, line 1
  // (note: line 1 is the second row, since counting begins with 0
  lcd.setCursor(0, 1);
  // print the number of seconds since reset:
  lcd.print(millis() / 1000);
}
```

Lab1: Task 3. Introducing the LCD display.

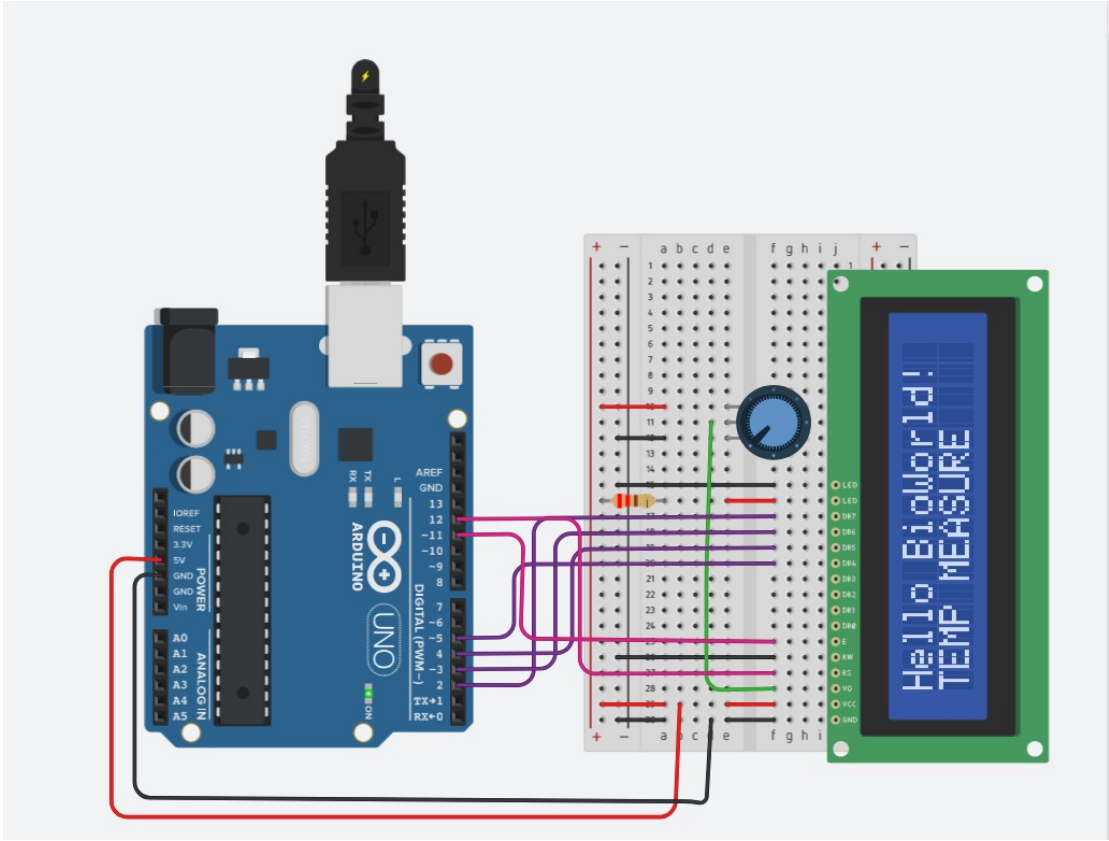


That task help us to understand how potentiometer affects directly over LCD Display. In fact, if we turn down the potentiometer, the range is 5V to 0V, it means less voltage, the display fade out the output text until disappear completely.

In the next task, we will interact with the temperature sensor and show the values related on the LCD display.

Following the first instructions, we need to adapt the circuit design and the code to get analog temperature, convert in digital value and show in the only two rows on the LCD display instead of Serial Monitor only.

Lab1: Task 3. Introducing the LCD display.



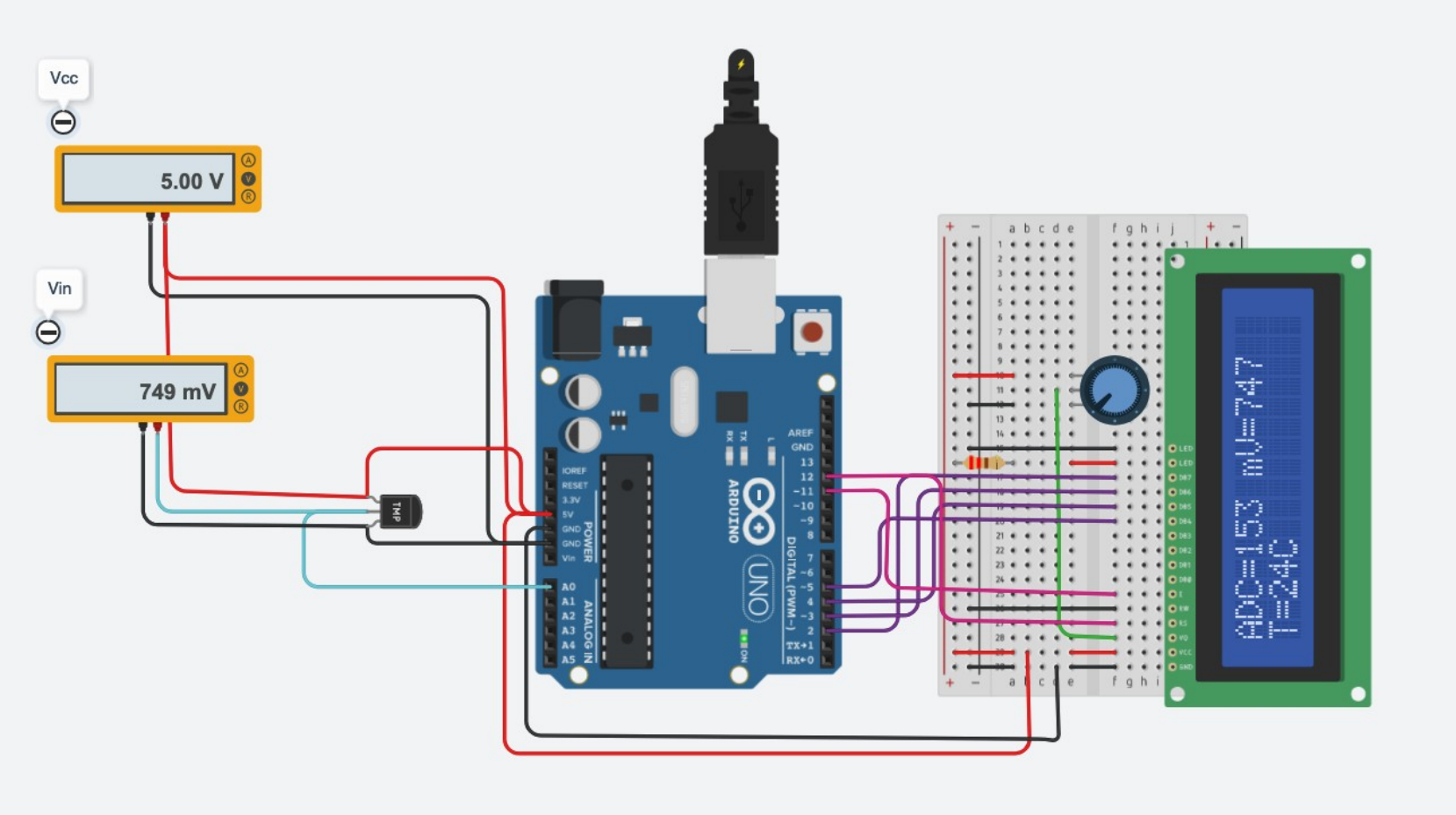

```
 * LCD VSS pin to ground
 * LCD VCC pin to 5V
 * 10K resistor:
 * ends to +5V and ground
 * wiper to LCD VO pin (pin 3)

 Library originally added 18 Apr 2008
 by David A. Mellis
 library modified 5 Jul 2009
 by Limor Fried (http://www.ladyada.net)
 example added 9 Jul 2009
 by Tom Igoe
 modified 22 Nov 2010
 by Tom Igoe
 modified 7 Nov 2016
 by Arturo Guadalupi

 This example code is in the public domain.

 https://docs.arduino.cc/learn/electronics/lcd-displays

*/

// include the library code:
#include <LiquidCrystal.h>

// initialize the library by associating any needed LCD interface
// with the arduino pin number it is connected to
const int rs = 12, en = 11, d4 = 5, d5 = 4, d6 = 3, d7 = 2;
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);

void setup() {
  lcdInit();
}

void loop() {
 delay(500);

}

void lcdInit(void) {
  lcd.begin(16,2);
    // Print a message to the LCD.
  lcd.print("Hello BioWorld! ");
  lcd.setCursor(0,1);
  lcd.print("TEMP MEASURE");
  delay(1000);
```

In the **task 3.2** we adapt the code to prepare the breadboard to join the temperature sensor and the LCD Display with welcoming message.

Then, we join the first task made before, with the temperature sensor in the left side and combine the code at once to show the values on LCD display instead on the Serial Monitor.

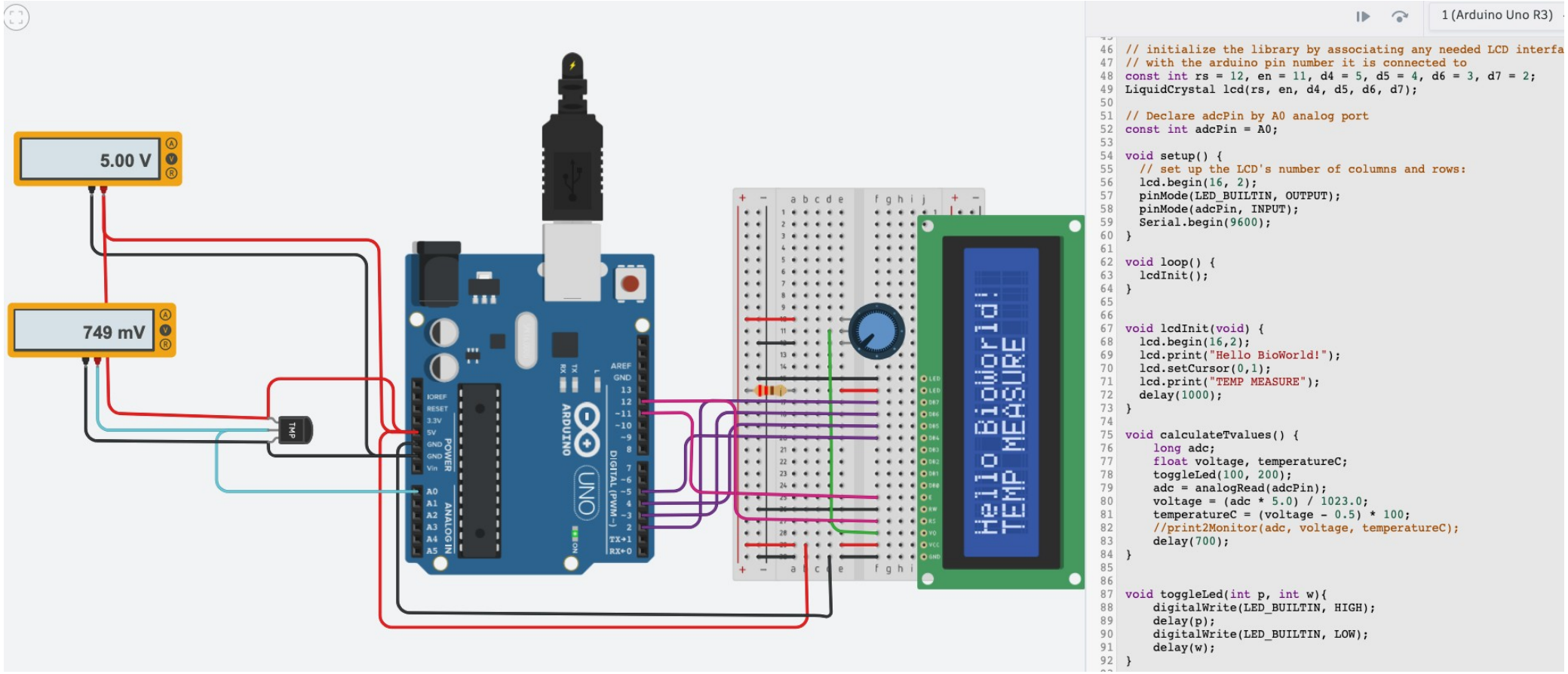
In this screenshot, we show the first adaption of the code.

Lab1: Task 3. Introducing the LCD display.



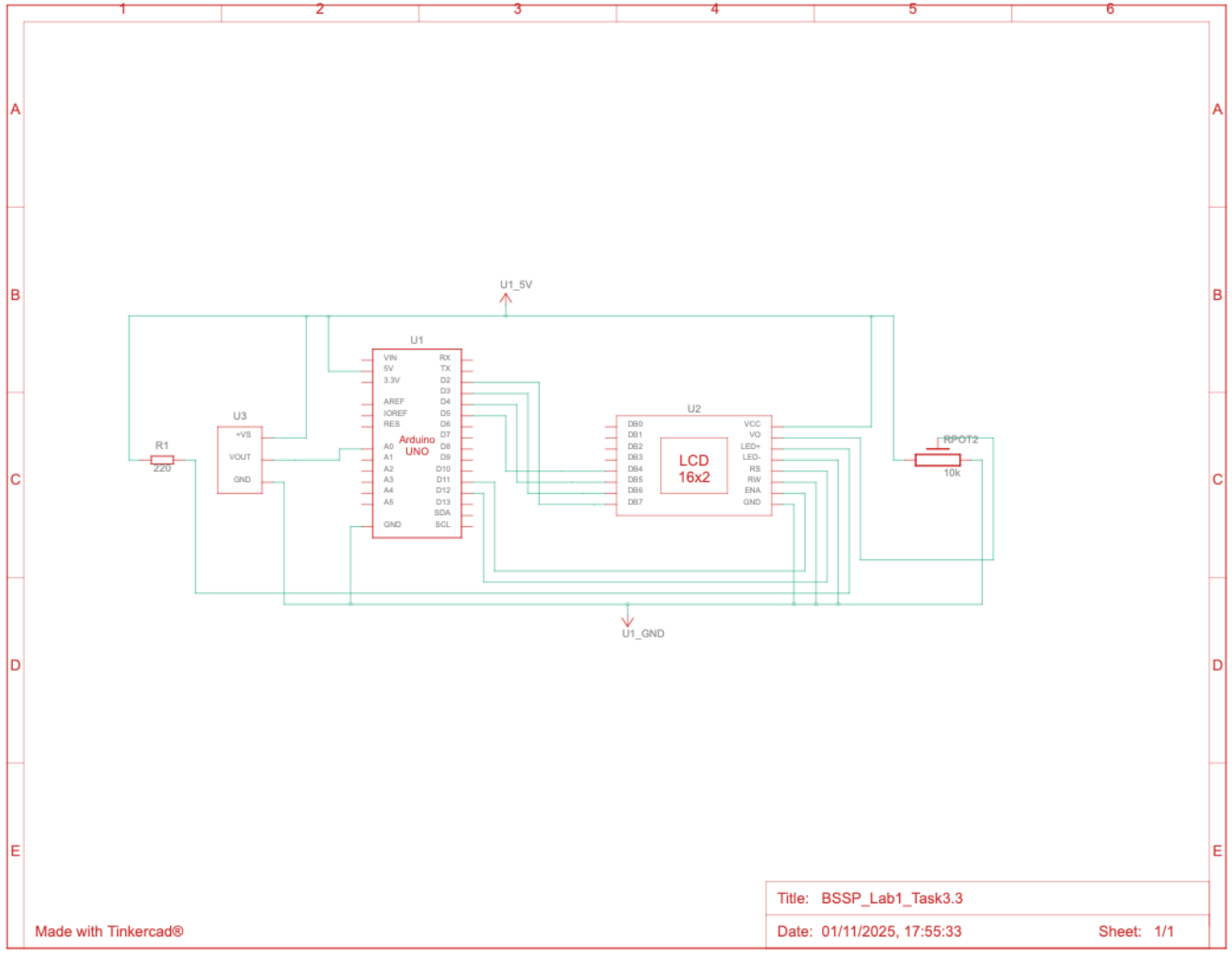
Finally, the **task 3.3** shows the values ADC; mV and T directly on LCD display instead on the Serial Monitor. In the next slide we will show the final schematic view and the code used to achieve this task.

Lab1: Task 3. Introducing the LCD display.



In this screenshot we join the first part, the temperature sensor shown in the task 2, and the right side with the LCD display.
In the next task 3.3 we will adapt the code by the function **calculateTvalues()** and send those values in the new function **lcdPrint()**

Lab1: Task 3. Introducing the LCD display.



Lab1: Task 3. Introducing the LCD display.

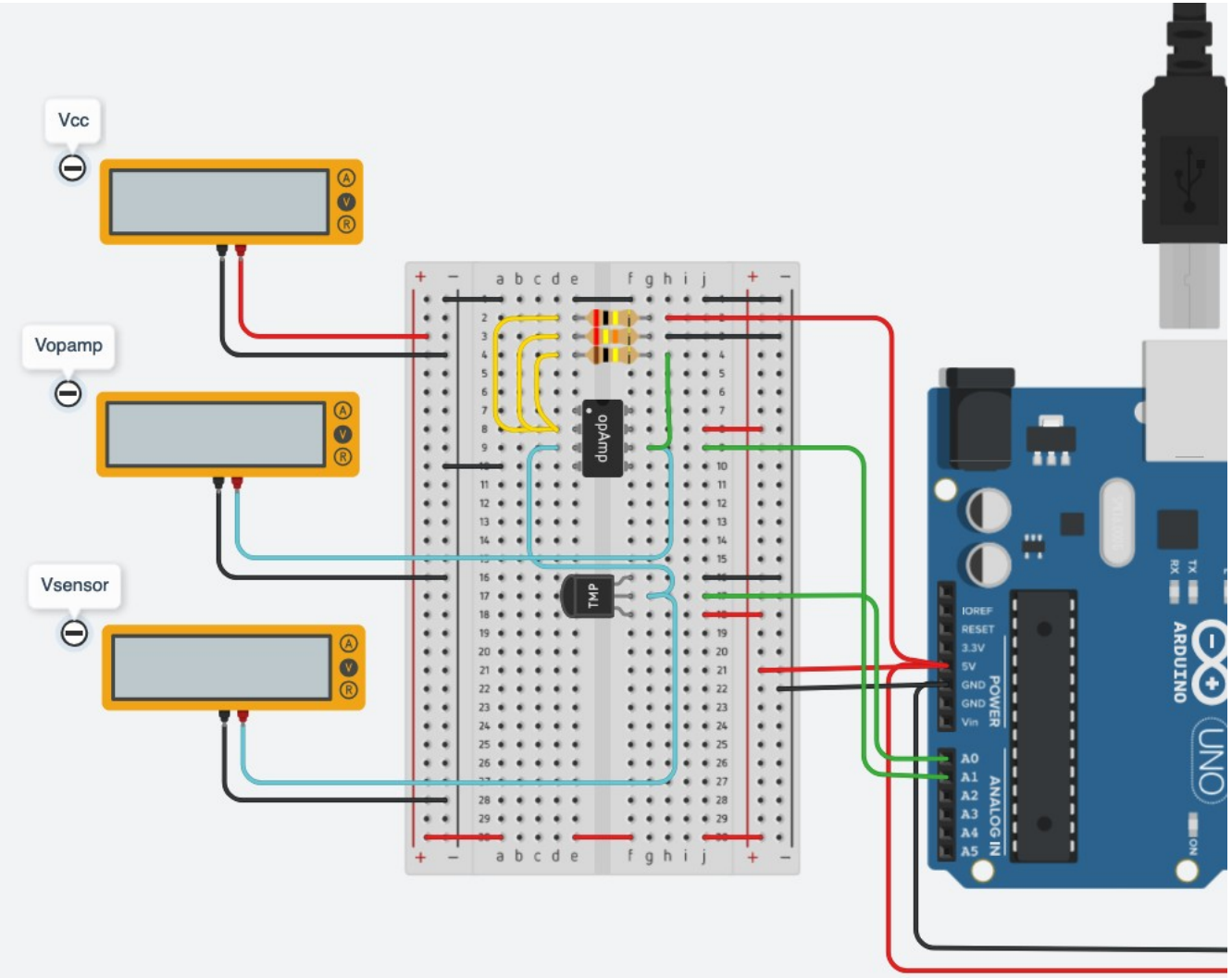
```
41  */
42
43  // include the library code:
44  #include <LiquidCrystal.h>
45
46  // initialize the library by associating any needed LCD interface
47  // with the arduino pin number it is connected to
48  const int rs = 12, en = 11, d4 = 5, d5 = 4, d6 = 3, d7 = 2;
49  LiquidCrystal lcd(rs, en, d4, d5, d6, d7);
50
51  // Declare adcPin by A0 analog port
52  const int adcPin = A0;
53
54  void setup() {
55    // set up the LCD's number of columns and rows:
56    lcd.begin(16, 2);
57    pinMode(LED_BUILTIN, OUTPUT);
58    pinMode(adcPin, INPUT);
59    Serial.begin(9600);
60  }
61
62  /* In the loop function call lcdInit
63      and the values calculated */
64  void loop() {
65    lcdInit();
66    calculateTvalues();
67  }
68
69  // Initialize the LCD display
70  void lcdInit(void) {
71    lcd.begin(16,2);
72    lcd.print("Hello BioWorld!");
73    lcd.setCursor(0,1);
74    lcd.print("TEMP MEASURE: ");
75    //Show the welcoming message for 1sec.
76    delay(1000);
77  }
78
```

```
79
80  // Main function to print LCD messages
81  void lcdPrint(long adc,
82                float voltage,
83                long temperatureC) {
84    // Clear the display first
85    lcd.clear();
86
87    // Set the 1st line for ADC and Voltage
88    lcd.setCursor(0, 0);
89    lcd.print("ADC=");
90    lcd.print(adc);
91    lcd.print(" ");
92    lcd.print("mV=");
93    lcd.print((int)(voltage * 1000));
94
95    // Set the 2nd line of Temperature
96    lcd.setCursor(0, 1);
97    lcd.print("T=");
98    lcd.print(temperatureC);
99    lcd.print("C");
100 }
101
102 // Main function to calculate values
103 void calculateTvalues() {
104   long adc;
105   float voltage;
106   long temperatureC;
107   adc = analogRead(adcPin);
108   voltage = (adc * 5.0) / 1023.0;
109   temperatureC = (voltage - 0.5) * 100;
110   lcdPrint(adc, voltage, temperatureC);
111   // Show the message during 5sec.
112   delay(5000);
113 }
114
```

Laboratory 1 Report

- Task 1 – Schematics of a mixed signal circuit.
- Task 2 – Programming the Arduino board.
- Task 3 – Introducing the LCD display.
- **Task 4 – Conditioning circuit.**
- Task 5 – Results.

Lab1: Task 4. Conditioning circuit.



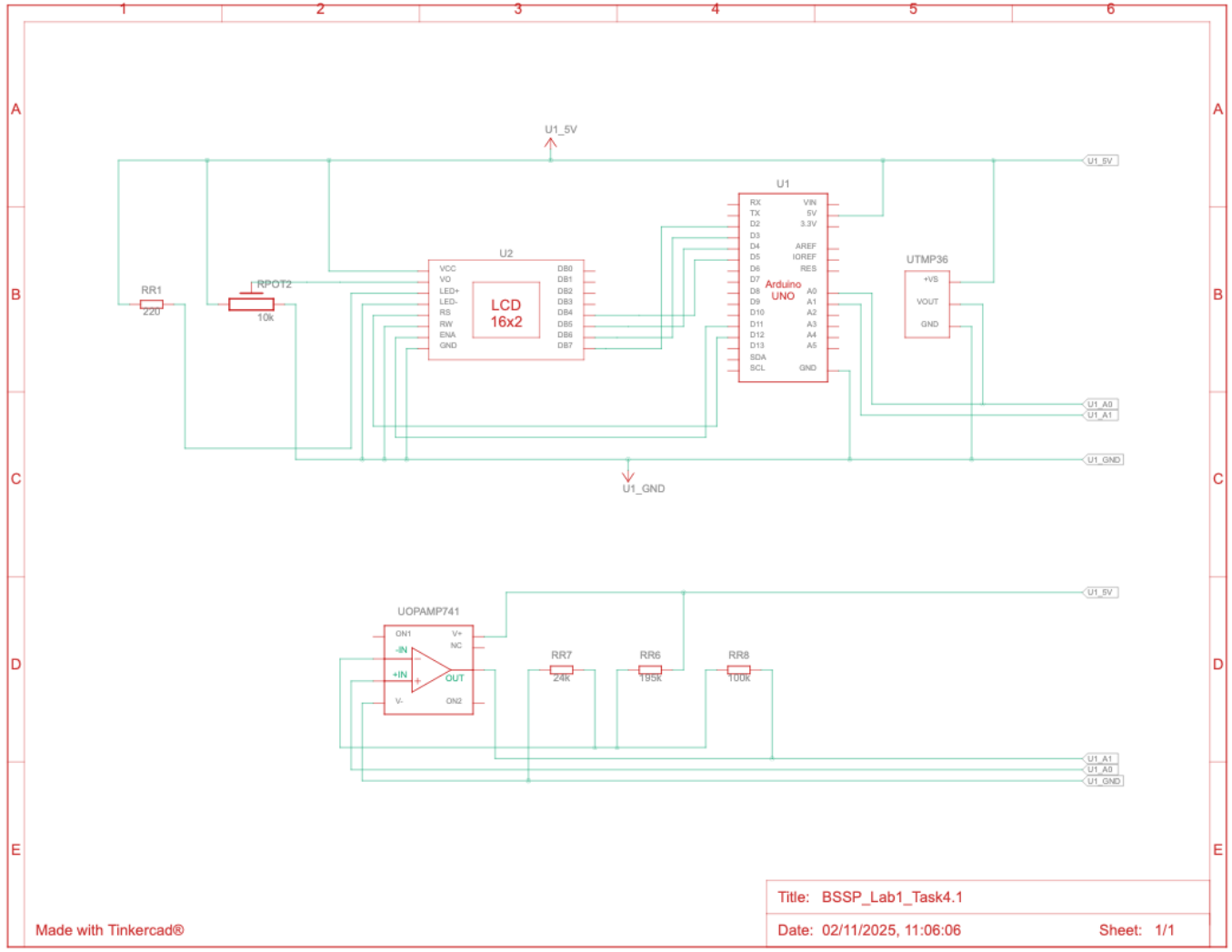
In the **task 4.1** we are going to design the new circuit following the requirements to optimise the previous circuit.

That optimisation involves the introduction of the **thermistor**. It maximizes the analogue input range into an ADC in order to maximize the resolution. In the same way, this task optimises the input of the ADC inside Arduino Uno from 0.25 to 4.75 V.

Taking the specifications of our Group E, the inlet temperatures to be considered are between 0 ºC and 80 ºC.

This input temperatures range must be **converted by the non-inverting amplifier** to an ADC voltage input between 0.25 V and 4.75 V.

Lab1: Task 4. Conditioning circuit.



In the **task 4.2** we show the schematic view with the solution corresponding the connections between the components.

To adjusting the frequency in the circuit, and re-create a thermistor, there are three resistors with default value assigned are 1kΩ. Resistors are used with a thermistor and Op-Amp to create a voltage divider, which converts the thermistor's resistance change into a voltage change. This voltage is then fed into the Op-Amp, which can be configured to amplify and linearise the signal, making the output proportional to temperature. Because of that, we need to calculate by the temperature range 0ºC and 80ºC and the power supply voltage, at 5V.

The second component is operation amplifier, opAmp741. It's important understand the pins: IN+ and IN- used for connecting the signal to be amplified or filtered; Power+ and Power- , respectively the input voltage and the ground.

To assign the values, apart the available academic material, the Unit3, we have checked the Arduino official documentation for OpAmp741 https://www.electronicshub.org/ic-741-op-amp-basics/

Lab1: Task 4. Conditioning circuit.

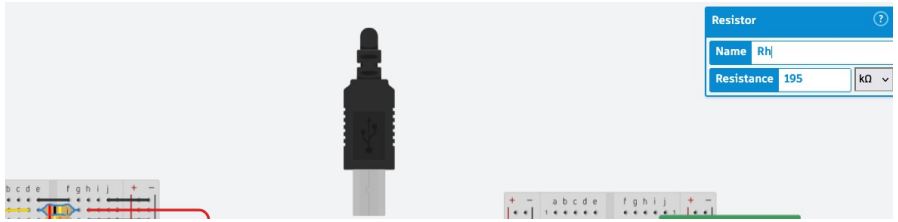
In the **task 4.3** we are going to calculate the resistors values.

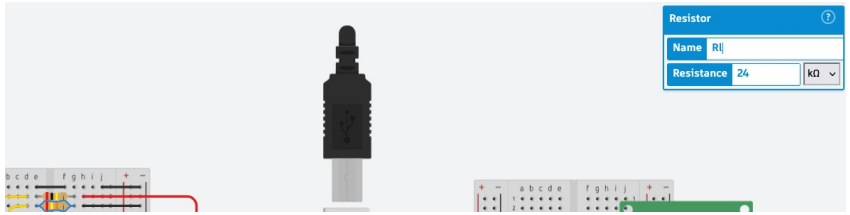
The first resistor, Rh which connect the inverting input, pin IN- from OpAmp and the power voltage 5V.
The second resistor, Rl connect between the inverting input pin IN- from OpAmp and the ground.
The third resistor, Rf connect between the inverting input pin IN- from OpAmp and the OpAmp ouput.

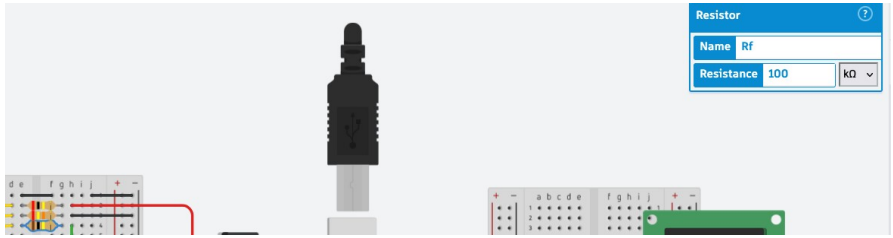
To calculate the resistor values, following the formula:

$$V_0 = V^+ \cdot (1 + \frac{Rf}{Rl} + \frac{Rf}{Rh}) \frac{Vr \cdot Rf}{Rh}$$

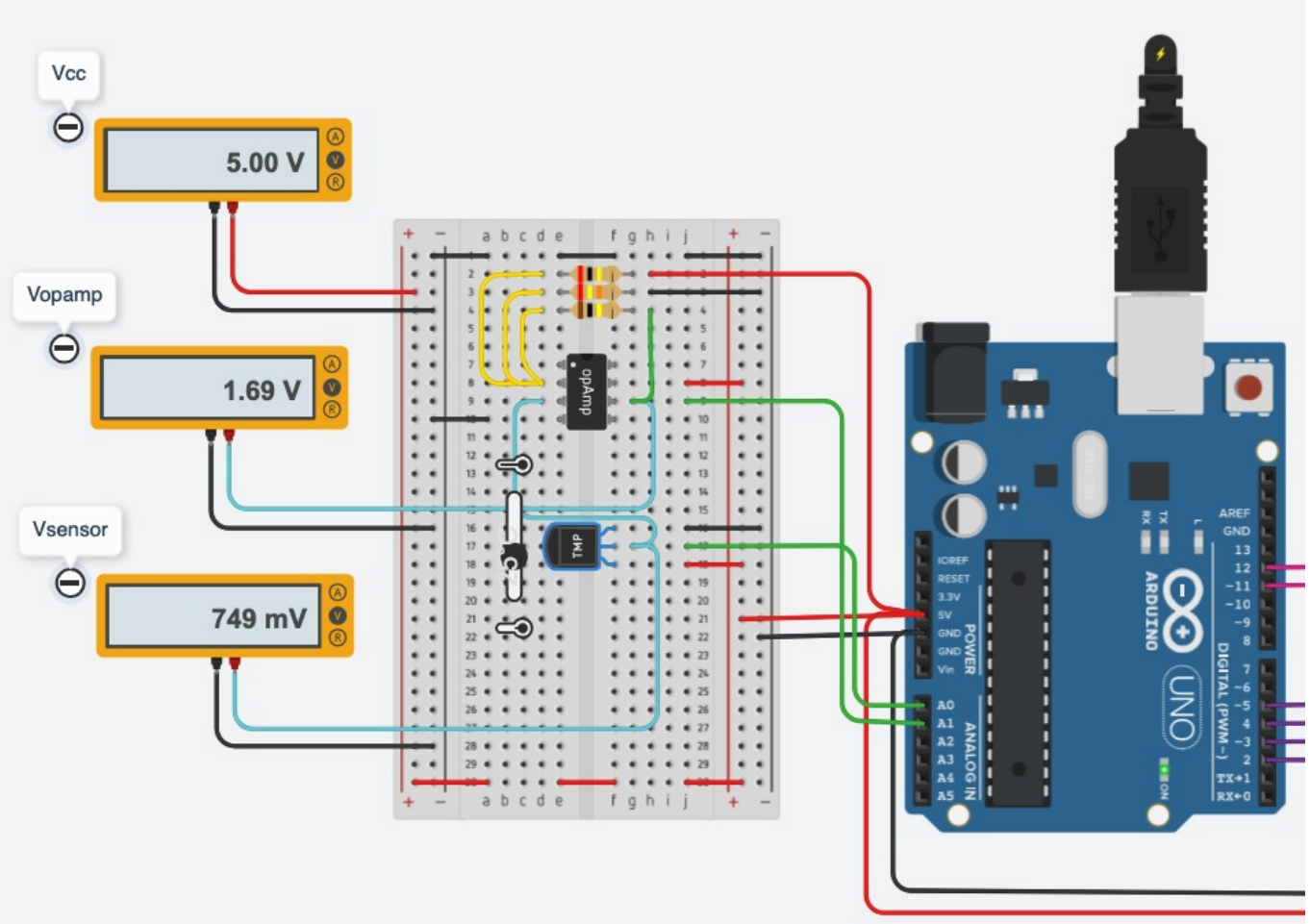
To calculate the offset, remember we have a temperature range from 0ºC to maximum 80ºC, the resultant values are:
- Rf = 100kΩ
- Rh = 195kΩ
- Rl = 24kΩ







Lab1: Task 4. Conditioning circuit.



In the **task 4.4** we show the values corresponding with the temperature set at 25ºC

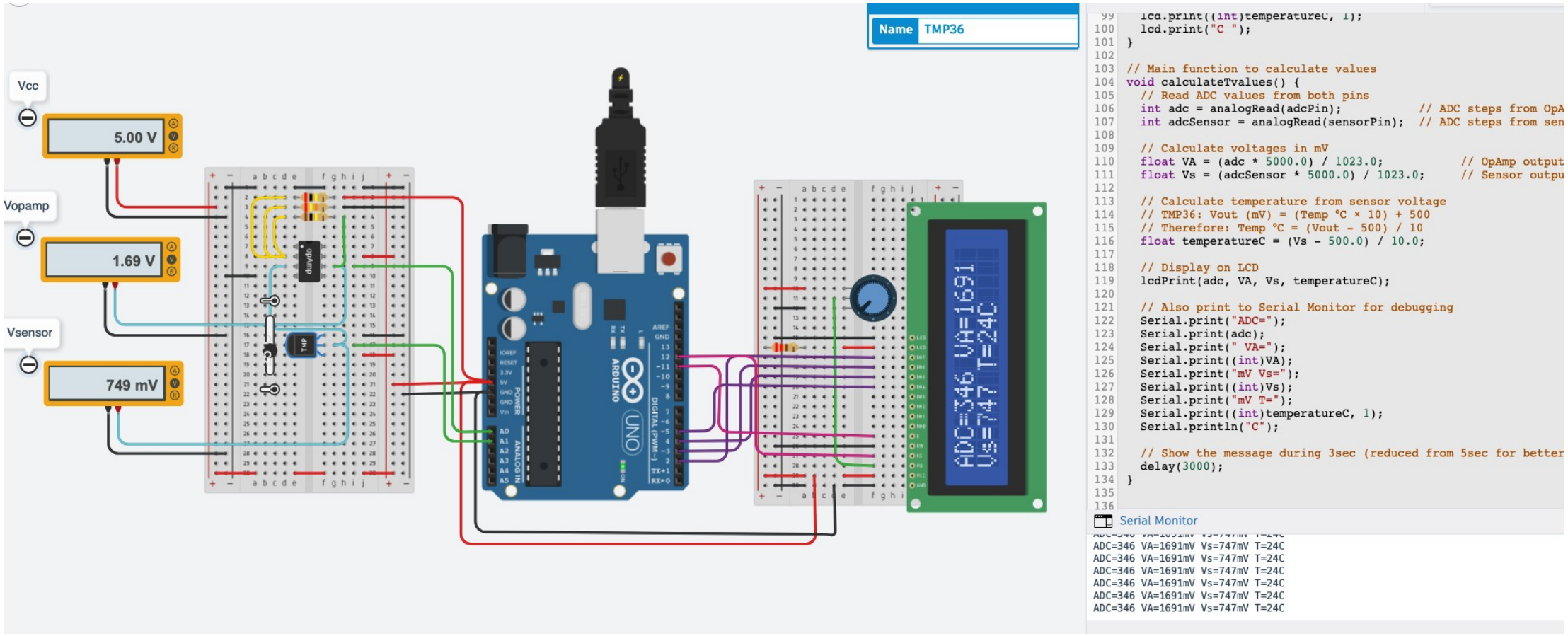
The Operational Amplifier send the corresponding value converted to voltmeter Vopamp, it shows 1.69V.

As expected the voltmeter for the temperature sensor, Vsensor shows 749mV.

Laboratory 1 Report

- Task 1 – Schematics of a mixed signal circuit.
- Task 2 – Programming the Arduino board.
- Task 3 – Introducing the LCD display.
- Task 4 – Conditioning circuit.
- **Task 5 – Results.**

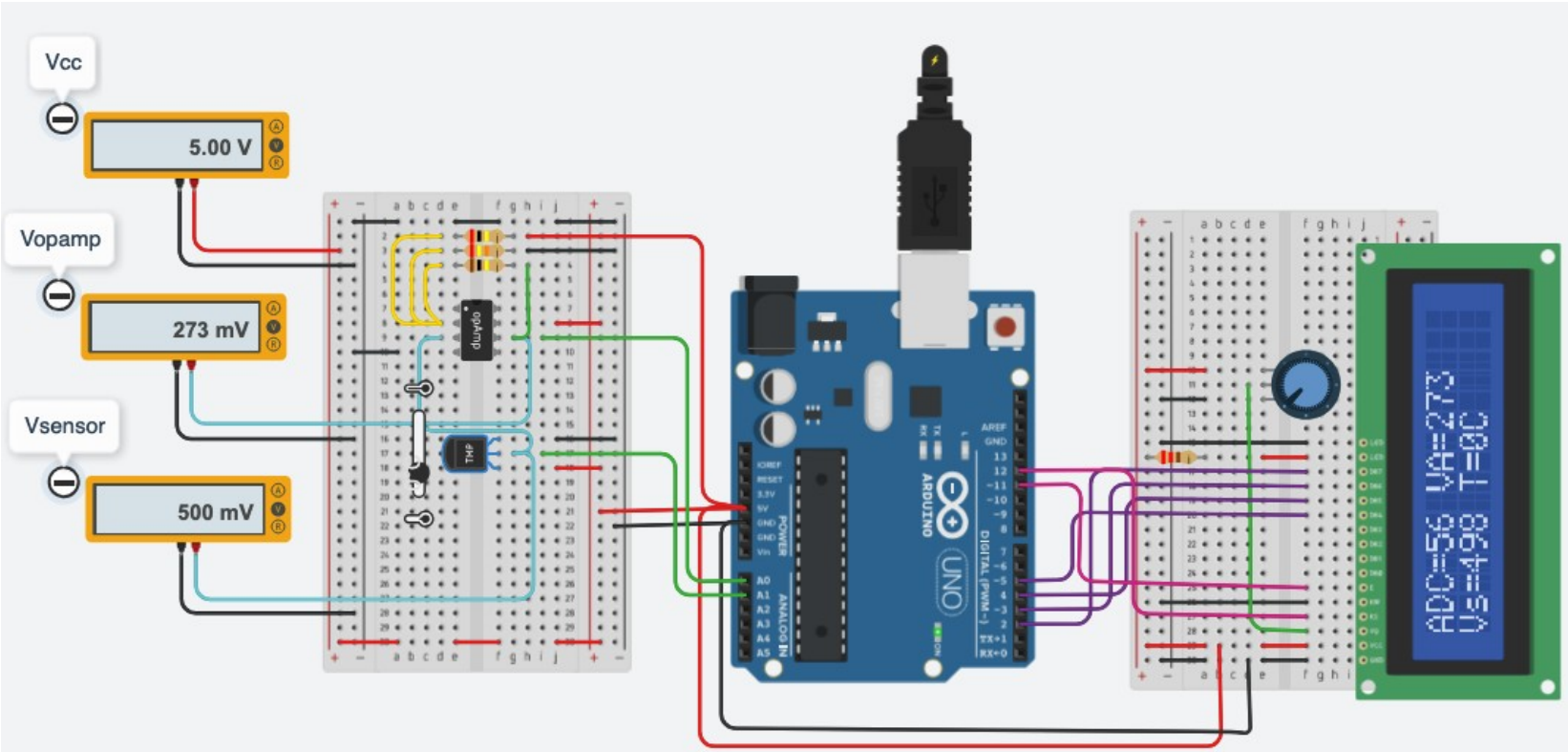
Lab1: Task 5. The sensor data acquisition system.




```
  lcd.print((int)temperatureC, 1);
  lcd.print("C ");
}

// Main function to calculate values
void calculateTvalues() {
  // Read ADC values from both pins
  int adc = analogRead(adcPin);            // ADC steps from OpA
  int adcSensor = analogRead(sensorPin);   // ADC steps from sen

  // Calculate voltages in mV
  float VA = (adc * 5000.0) / 1023.0;          // OpAmp output
  float Vs = (adcSensor * 5000.0) / 1023.0;    // Sensor outpu

  // Calculate temperature from sensor voltage
  // TMP36: Vout (mV) = (Temp °C × 10) + 500
  // Therefore: Temp °C = (Vout - 500) / 10
  float temperatureC = (Vs - 500.0) / 10.0;

  // Display on LCD
  lcdPrint(adc, VA, Vs, temperatureC);

  // Also print to Serial Monitor for debugging
  Serial.print("ADC=");
  Serial.print(adc);
  Serial.print(" VA=");
  Serial.print((int)VA);
  Serial.print("mV Vs=");
  Serial.print((int)Vs);
  Serial.print("mV T=");
  Serial.print((int)temperatureC, 1);
  Serial.println("C");

  // Show the message during 3sec (reduced from 5sec for better
  delay(3000);
}
```

In the **task 5** we will put all together and show the values requested on LCD display and on the Serial Monitor. In the next slides we will show the entire code used and the schematic view. Also, we fill the table V with the requested values.

Lab1: Task 5. The sensor data acquisition system.



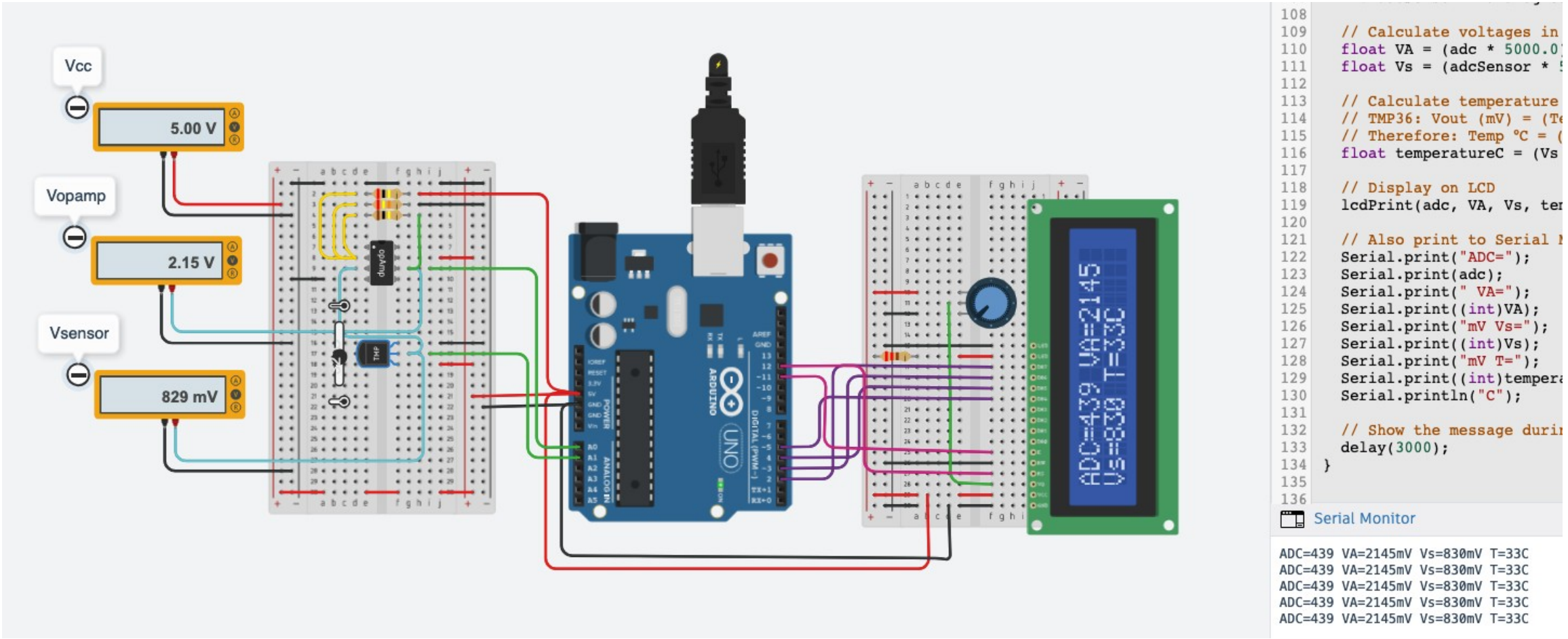

```
108
109   // Calculate voltages in mV
110   float VA = (adc * 5000.0) / 1023.0;
111   float Vs = (adcSensor * 5000.0) / 1023.
112
113   // Calculate temperature from sensor vo
114   // TMP36: Vout (mV) = (Temp °C × 10) +
115   // Therefore: Temp °C = (Vout - 500) /
116   float temperatureC = (Vs - 500.0) / 10.
117
118   // Display on LCD
119   lcdPrint(adc, VA, Vs, temperatureC);
120
121   // Also print to Serial Monitor for deb
122   Serial.print("ADC=");
123   Serial.print(adc);
124   Serial.print(" VA=");
125   Serial.print((int)VA);
126   Serial.print("mV Vs=");
127   Serial.print((int)Vs);
128   Serial.print("mV T=");
129   Serial.print((int)temperatureC, 1);
130   Serial.println("C");
131
132   // Show the message during 3sec (reduce
133   delay(3000);
134 }
135
136
```

Serial Monitor

```
ADC=346 VA=1691mV Vs=747mV T=24C
ADC=346 VA=1691mV Vs=747mV T=24C
ADC=253 VA=1236mV Vs=669mV T=16C
ADC=56 VA=273mV Vs=498mV T=0C
ADC=56 VA=273mV Vs=498mV T=0C
ADC=56 VA=273mV Vs=498mV T=0C
```

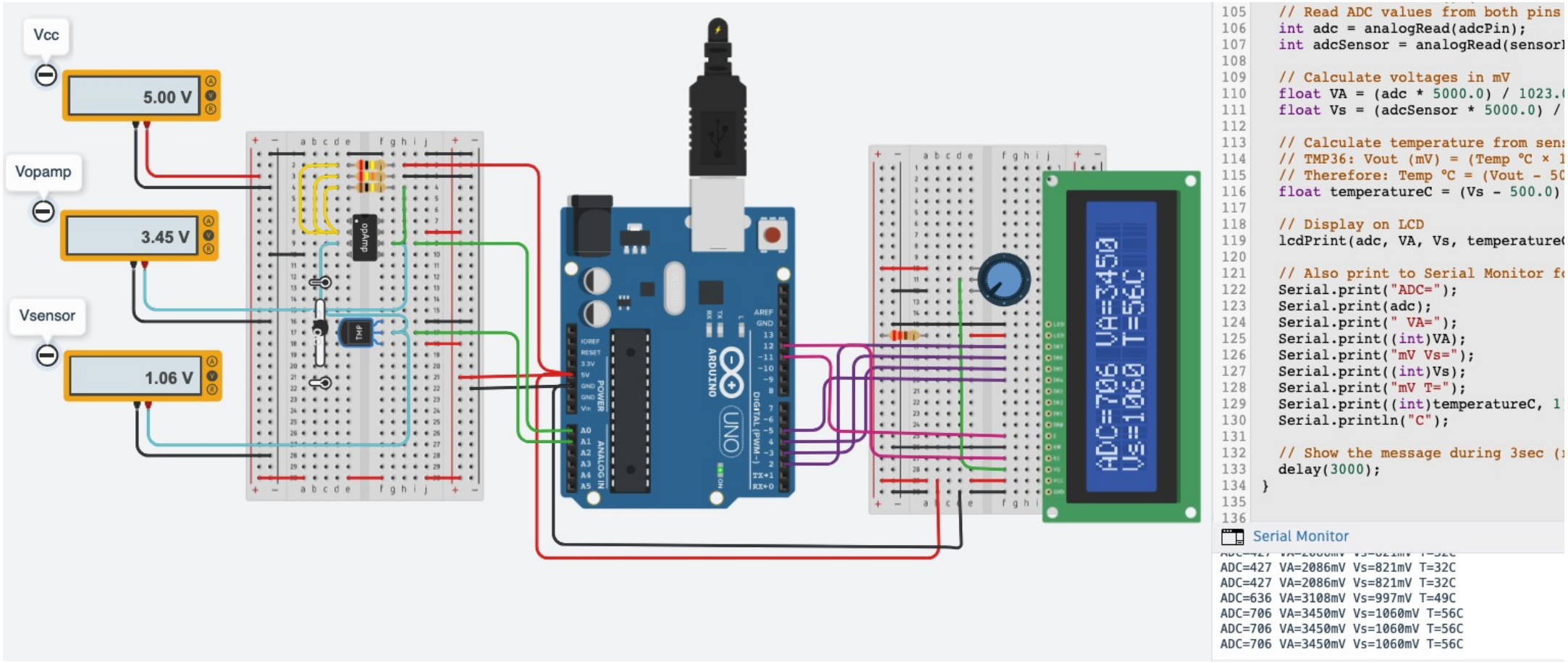
In the screenshot the temperature Tmin has been configured at 0ºC with the following values: ADC=56; VA=273mV; Vs=498mV.

Lab1: Task 5. The sensor data acquisition system.



In the screenshot the temperature T1 has been configured at 33ºC with the following values: ADC=439; VA=2145mV; Vs=830mV.

Lab1: Task 5. The sensor data acquisition system.



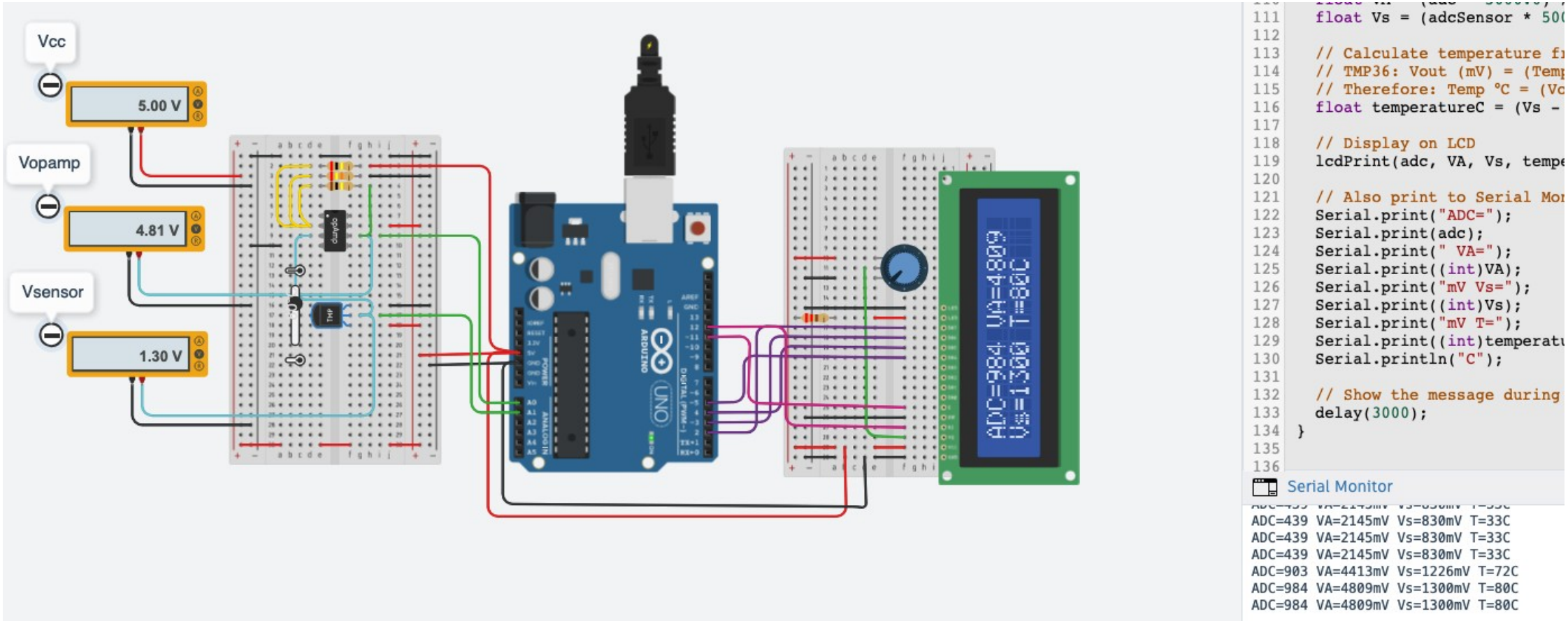

```
105   // Read ADC values from both pins
106   int adc = analogRead(adcPin);
107   int adcSensor = analogRead(sensor
108
109   // Calculate voltages in mV
110   float VA = (adc * 5000.0) / 1023.
111   float Vs = (adcSensor * 5000.0) /
112
113   // Calculate temperature from sens
114   // TMP36: Vout (mV) = (Temp °C × 1
115   // Therefore: Temp °C = (Vout - 50
116   float temperatureC = (Vs - 500.0)
117
118   // Display on LCD
119   lcdPrint(adc, VA, Vs, temperature(
120
121   // Also print to Serial Monitor fo
122   Serial.print("ADC=");
123   Serial.print(adc);
124   Serial.print(" VA=");
125   Serial.print((int)VA);
126   Serial.print("mV Vs=");
127   Serial.print((int)Vs);
128   Serial.print("mV T=");
129   Serial.print((int)temperatureC, 1
130   Serial.println("C");
131
132   // Show the message during 3sec (1
133   delay(3000);
134 }
135
136
```

Serial Monitor

```
ADC=427 VA=2086mV Vs=821mV T=32C
ADC=427 VA=2086mV Vs=821mV T=32C
ADC=636 VA=3108mV Vs=997mV T=49C
ADC=706 VA=3450mV Vs=1060mV T=56C
ADC=706 VA=3450mV Vs=1060mV T=56C
ADC=706 VA=3450mV Vs=1060mV T=56C
```

In the screenshot the temperature T2 has been configured at 56ºC with the following values: ADC=706; VA=3450mV; Vs=1060mV.

Lab1: Task 5. The sensor data acquisition system.



In the screenshot the temperature Tmax has been configured at 80ºC with the following values: ADC=984; VA=4809mV; Vs=1300mV.

Lab1: Task 5. The sensor data acquisition system.

```ino
// include the library code:
#include <LiquidCrystal.h>

// initialize the library by associating any needed LCD interface pin
// with the arduino pin number it is connected to
const int rs = 12, en = 11, d4 = 5, d5 = 4, d6 = 3, d7 = 2;
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);

// Declare adcPin by A0 analog port
const int adcPin = A0;
const int sensorPin = A1;

void setup() {
  // set up the LCD's number of columns and rows:
  lcd.begin(16, 2);
  pinMode(LED_BUILTIN, OUTPUT);
  pinMode(adcPin, INPUT);
  pinMode(sensorPin, INPUT);
  Serial.begin(9600);
}

/* In the loop function call lcdInit
   and the values calculated */
void loop() {
  lcdInit();
  calculateTvalues();
}

// Initialize the LCD display
void lcdInit(void) {
  lcd.begin(16, 2);
  lcd.print("Hello BioWorld!");
  lcd.setCursor(0, 1);
  lcd.print("TEMP MEASURE");
  // Show the welcoming message for 1sec.
  delay(1000);
}
```

Serial Monitor

```ino
// Main function to print LCD messages
void lcdPrint(int adc, float VA, float Vs, float temperatureC) {
  // Clear the display first
  lcd.clear();

  // Line 1: ADC steps and VA (OpAmp output in mV)
  lcd.setCursor(0, 0);
  lcd.print("ADC=");
  lcd.print(adc);
  lcd.print(" VA=");
  lcd.print((int)VA);
  lcd.print("   ");

  // Line 2: Vs (sensor output in mV) and Temperature
  lcd.setCursor(0, 1);
  lcd.print("Vs=");
  lcd.print((int)Vs);
  lcd.print(" T=");
  lcd.print((int)temperatureC, 1);
  lcd.print("C ");
}

// Main function to calculate values
void calculateTvalues() {
  // Read ADC values from both pins
  int adc = analogRead(adcPin);           // ADC steps from OpAmp output (A0)
  int adcSensor = analogRead(sensorPin);  // ADC steps from sensor (A1)

  // Calculate voltages in mV
  float VA = (adc * 5000.0) / 1023.0;           // OpAmp output voltage (mV)
  float Vs = (adcSensor * 5000.0) / 1023.0;     // Sensor output voltage (mV)

  // Calculate temperature from sensor voltage
  // TMP36: Vout (mV) = (Temp °C × 10) + 500
  // Therefore: Temp °C = (Vout - 500) / 10
  float temperatureC = (Vs - 500.0) / 10.0;

  // Display on LCD
```

Lab1: Task 5. The sensor data acquisition system.

```
102
103 // Main function to calculate values
104 void calculateTvalues() {
105   // Read ADC values from both pins
106   int adc = analogRead(adcPin);           // ADC steps from OpAmp output (A0)
107   int adcSensor = analogRead(sensorPin);  // ADC steps from sensor (A1)
108
109   // Calculate voltages in mV
110   float VA = (adc * 5000.0) / 1023.0;           // OpAmp output voltage (mV)
111   float Vs = (adcSensor * 5000.0) / 1023.0;     // Sensor output voltage (mV)
112
113   // Calculate temperature from sensor voltage
114   // TMP36: Vout (mV) = (Temp °C × 10) + 500
115   // Therefore: Temp °C = (Vout - 500) / 10
116   float temperatureC = (Vs - 500.0) / 10.0;
117
118   // Display on LCD
119   lcdPrint(adc, VA, Vs, temperatureC);
120
121   // Also print to Serial Monitor for debugging
122   Serial.print("ADC=");
123   Serial.print(adc);
124   Serial.print(" VA=");
125   Serial.print((int)VA);
126   Serial.print("mV Vs=");
127   Serial.print((int)Vs);
128   Serial.print("mV T=");
129   Serial.print((int)temperatureC, 1);
130   Serial.println("C");
131
132   // Show the message during 3sec
133   delay(3000);
134 }
135
136
```

Lab1: Task 5. The sensor data acquisition system.

	T (ºC)	Vout Sensor (V)	Vout OpAmp (V)	ADC counts (LCD)	OpAmp (LCD)	Sensor Out (LCD)	Measured T in LCD	Error %
Minimum Temp. value	0ºC	500mV	273mV	56	273mV	498mV	0ºC	0.4%
T1	32ºC	829mV	2.15V	439	2145mV	830mV	33ºC	-0.12%
T2	56ºC	1.06V	3.45V	706	3450mV	1060mV	56ºC	0%
Maximum Temp. value	80ºC	1.30V	4.81V	984	4809mV	1300mV	80ºC	0%

Table V – It's a final table where we consider the 8 values between the minimum and maximum input sensor range. Finally, we calculate the error% between sensor value and sensor output shown in LCD display value.

Final evaluation of Laboratory 1

This laboratory practice successfully demonstrated the implementation of a complete temperature sensing system, from the beginning using the simple potentiometer and show the output on the voltmeter. Calculating and convert the values using other components such as the TMP36 sensor, OpAmp 741 conditioning circuit, and Arduino Uno microcontroller.

Key achievements learnt:

- Designed and implemented a non-inverting amplifier circuit that optimised the ADC input range from 0.25V to 4.75V for the 0-80°C temperature range.

- Calculated optimal resistor values (Rf=100kΩ, Rl=24kΩ, Rh=195kΩ) achieving a gain of 5.6 and offset of 2.56V

- Successfully integrated LCD display to show ADC steps, amplified voltage (VA), sensor voltage (Vs), and calculated temperature

- Improved measurement resolution from 0.405°C/step to 0.15°C/step through proper signal conditioning

The practice reinforced understanding of analog signal conditioning, ADC conversion principles, and the importance of maximising input voltage span for better measurement accuracy. Finally, the hands-on experience with Tinkercad simulation validated theoretical calculations and demonstrated practical circuit design optimisation techniques essential in biomedical instrumentation.

Marco Russo
marco.russo@estudiants.urv.cat

Final evaluation of Laboratory 1

Final evaluation of Laboratory 1

Final evaluation of Laboratory 1

Thank you for your attention!

MHEDAS
MASTER HEALTH DATA SCIENCE

- Contact

  - Carmen Aznar Mathonneau
  - Yuxi Qiao
  - Marco Russo -
  - Kamela Xhengo